

# Coding Interview Preparation Guide — Volume 2

---

## Deeper dive: Binary Search, Tries, and Dynamic Programming (Java 8+)

**Target level:** Mid-level software engineering interviews at companies such as Booking.com, Airbnb, Microsoft, Agoda, UiPath, and organizations of similar range.

---

## About this volume

Volume 1 covered the twelve core patterns with a broad set of problems. **Volume 2 goes deeper** into the three areas that separate a "can code" candidate from a "thinks in patterns" one — and the three you asked to focus on:

- **Binary Search** — beyond "find x in a sorted array." The real skill is spotting a *monotonic answer space* and binary-searching over it (Split Array, Ship Packages).
- **Tries (prefix trees)** — the go-to structure whenever a problem says "prefix", "starts with", "dictionary", or "autocomplete".
- **Dynamic Programming** — the classic grid, string, and knapsack recurrences that show up again and again.

Every problem is **new** (nothing repeated from Volume 1) and follows the same layout: plain-language concept → worked example → ASCII diagram → brute force → optimized Java → complexity table. All code targets **Java 8+** and compiles as-is.

## What's inside

| # | Section                        | Problems | Why it matters                                            |
|---|--------------------------------|----------|-----------------------------------------------------------|
| 1 | Binary Search (advanced)       | 7        | "Search on the answer" is a top-tier signal               |
| 2 | Tries                          | 5        | Prefix/dictionary problems have a dead-giveaway structure |
| 3 | Dynamic Programming (advanced) | 8        | Grid / string / knapsack recurrences                      |
| 4 | Mixed high-frequency set       | 4        | Fresh must-know problems across patterns                  |

## The mental checklist (works for all three areas)

1. **Binary search?** Is the data sorted, OR can I phrase the answer as "smallest/largest value X for which a yes/no test flips from false to true"? If yes, binary-search X.
2. **Trie?** Does the problem repeatedly ask about **prefixes** or membership in a word list? Build a trie once, query many times.
3. **DP?** Can I define the answer for input  $n$  in terms of smaller inputs, and do those subproblems repeat? Write the **recurrence in words first**, then translate to a table.



# 1. Binary Search (Advanced)

---

**The core idea.** Classic binary search finds a value inside a sorted array. The more powerful version searches over a **space of candidate answers** — capacities, sums, distances — instead of array indices. You guess a candidate, run a cheap yes/no feasibility check on it, and lean on the fact that the check is **monotonic** (once it flips from "no" to "yes" it stays "yes" as the candidate grows, or vice versa) to throw away half the remaining candidates every time. This is called **binary search on the answer space**. The hard part is rarely the binary search loop itself; it's spotting the monotonic predicate and writing the feasibility check.

**Reach for it when:** the search space is sorted (or rotated-but-still-ordered), or is a **range of possible numeric answers** where you can cheaply test "is this value good enough?" and that test flips exactly once from false to true (or true to false).

---

## 1.1 Search a 2D Matrix (LC 74)

**Problem.** Given an  $m \times n$  integer matrix and an integer `target`, return `true` if `target` exists anywhere in the matrix and `false` otherwise. The matrix has two special properties: every row is sorted in ascending order, and the first integer of every row is greater than the last integer of the previous row (so the matrix, read row by row, is one long sorted sequence).

- **Constraints:**  $1 \leq m, n \leq 100$ ;  $-10^4 \leq \text{matrix}[i][j], \text{target} \leq 10^4$ .
- **Clarifying assumptions:** the matrix is non-empty and every row has the same number of columns; there may or may not be duplicate values, but that doesn't affect this search since we only need existence, not position.
- **Why it's tricky:** it looks like a 2D search, which tempts candidates into a staircase walk or a full scan, but the row-linking property secretly makes it a single sorted 1D array — the only real work is mapping a flat index back to `(row, col)`.

**Example.**

```
Input: matrix = [[1,3,5,7],
                 [10,11,16,20],
                 [23,30,34,60]], target = 3
Output: true
```

**Intuition (diagram).** Because each row picks up exactly where the previous row left off, the whole grid is really one sorted 1D array in disguise. Map a single flat index `mid` (from `0` to `rows*cols - 1`) to `(row, col) = (mid / cols, mid % cols)` and binary search as if it were a plain array. (A "staircase" search from the top-right corner also works, but flattening is simpler to reason about.)

|           |   |   |   |   |    |    |    |    |    |    |    |    |
|-----------|---|---|---|---|----|----|----|----|----|----|----|----|
| flat idx: | 0 | 1 | 2 | 3 | 4  | 5  | 6  | 7  | 8  | 9  | 10 | 11 |
| value:    | 1 | 3 | 5 | 7 | 10 | 11 | 16 | 20 | 23 | 30 | 34 | 60 |

```

cols = 4      row = idx / cols      col = idx % cols
e.g. idx = 6  ->  row = 1, col = 2  ->  value 16

```

## Approach (step by step)

- **Key idea / invariant:** treat the matrix as a single sorted array of length `rows * cols`. `lo` and `hi` always bound the flat-index range that could still contain `target`; every iteration halves that range exactly like a plain binary search.
- Set `lo = 0`, `hi = rows*cols - 1`.
- While `lo <= hi`: compute `mid = lo + (hi - lo) / 2` (a flat index).
- Convert `mid` back to a cell with `row = mid / cols`, `col = mid % cols`, and read `matrix[row][col]`.
- If it equals `target`, return `true`. If it's smaller, the target must be further along the flattened array, so `lo = mid + 1`. If it's larger, `hi = mid - 1`.
- If the loop ends without a match, `target` isn't in the matrix — return `false`.
- **Worked trace** (target = 3): `lo=0, hi=11 -> mid=5 -> (1,1)=11 > 3 -> hi=4 -> mid=2 -> (0,2)=5 > 3 -> hi=1 -> mid=0 -> (0,0)=1 < 3 -> lo=1 -> mid=1 -> (0,1)=3 -> match, return true`.

## Brute force

Scan every cell.

```

public boolean searchMatrixBrute(int[][] matrix, int target) {
    for (int[] row : matrix) {
        for (int val : row) {
            if (val == target) return true;
        }
    }
    return false;
}

```

## Optimized (binary search on flattened index)

Treat the grid as one sorted array of size `rows * cols`.

```

public boolean searchMatrix(int[][] matrix, int target) {
    int rows = matrix.length, cols = matrix[0].length;
    int lo = 0, hi = rows * cols - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;           // safe midpoint
        int val = matrix[mid / cols][mid % cols]; // map flat index -> (row, col)
        if (val == target) {
            return true;
        } else if (val < target) {
            lo = mid + 1;
        } else {
            hi = mid - 1;
        }
    }
    return false;
}

```

| Version       | Time                 | Space  |
|---------------|----------------------|--------|
| Brute force   | $O(m \cdot n)$       | $O(1)$ |
| Binary search | $O(\log(m \cdot n))$ | $O(1)$ |

## 1.2 Find First and Last Position of Element in Sorted Array (LC 34)

**Problem.** Given an array of integers `nums` sorted in non-decreasing order and an integer `target`, return the starting and ending index of `target`'s contiguous run of occurrences. Return `[-1, -1]` if `target` doesn't appear.

- **Constraints:**  $0 \leq \text{nums.length} \leq 10^5$ ;  $-10^9 \leq \text{nums}[i]$ ,  $\text{target} \leq 10^9$ ; `nums` is sorted ascending and may contain duplicates.
- **Clarifying assumptions:** all occurrences of `target` are contiguous (guaranteed by the sort); the array can be empty, which should immediately yield `[-1, -1]`.
- **Why it's tricky:** a single binary search only finds *one* occurrence, and which one it lands on is undefined when there are duplicates — you need two independently tie-broken searches, one biased left and one biased right.

**Example.**

```

Input:  nums = [5,7,7,8,8,10], target = 8
Output: [3, 4]

```

**Intuition (diagram).** Run two separate binary searches with different tie-breaking rules: one that keeps narrowing **leftward** on a match (to find the first occurrence), and one that keeps narrowing **rightward** on a match (to find the last). Each is still a plain binary search — only the "what to do when I find the target" branch changes.

```
idx: 0 1 2 3 4 5
```

```
val: 5 7 7 8 8 10
```

```
l r          target = 8, first match at idx 3, last match at idx 4
```

leftmost search: on a match, shrink  $hi = mid - 1$  → keeps sliding left → lands on 3

rightmost search: on a match, shrink  $lo = mid + 1$  → keeps sliding right → lands on 4

## Approach (step by step)

- **Key idea / invariant:** `findBound` is an ordinary binary search that additionally records the index of every match it sees in `result`, then deliberately keeps searching past it — left ( $hi = mid - 1$ ) to hunt for an earlier occurrence, or right ( $lo = mid + 1$ ) to hunt for a later one. The last recorded `result` before the loop ends is the correct bound.
- Call `findBound(nums, target, true)` to get the first index: on `nums[mid] == target`, save `mid` as the best-so-far answer and continue searching the **left** half.
- Call `findBound(nums, target, false)` to get the last index: on a match, save `mid` and continue searching the **right** half.
- If `target` never matches, `result` stays `-1` for that call.
- **Worked trace** (`target = 8, findFirst = true`): `lo=0, hi=5` → `mid=2, val=7 < 8` → `lo=3` → `mid=4, val=8` match, `result=4`, `hi=3` → `mid=3, val=8` match, `result=3`, `hi=2` → loop ends (`lo=3 > hi=2`) → returns `3`.

## Brute force

Linear scan, remembering the first and last match.

```
public int[] searchRangeBrute(int[] nums, int target) {
    int first = -1, last = -1;
    for (int i = 0; i < nums.length; i++) {
        if (nums[i] == target) {
            if (first == -1) first = i;
            last = i;
        }
    }
    return new int[] { first, last };
}
```

## Optimized (two bounded binary searches)

```
public int[] searchRange(int[] nums, int target) {
    return new int[] { findBound(nums, target, true), findBound(nums, target, false) };
}

private int findBound(int[] nums, int target, boolean findFirst) {
    int lo = 0, hi = nums.length - 1, result = -1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;          // safe midpoint
        if (nums[mid] < target) {
            lo = mid + 1;
        } else if (nums[mid] > target) {
            hi = mid - 1;
        } else {
            result = mid;                      // record the match, keep narrowing
            if (findFirst) {
                hi = mid - 1;                  // look further left for an earlier one
            } else {
                lo = mid + 1;                  // look further right for a later one
            }
        }
    }
    return result;
}
```

| Version             | Time        | Space  |
|---------------------|-------------|--------|
| Brute force         | $O(n)$      | $O(1)$ |
| Two binary searches | $O(\log n)$ | $O(1)$ |

## 1.3 Find Peak Element (LC 162)

**Problem.** Given an integer array `nums`, find a peak element and return its index. An element is a peak if it is strictly greater than both of its neighbors; treat positions outside the array bounds as `-infinity` (so the first/last element only needs to beat its one real neighbor). If multiple peaks exist, return the index of **any** one of them, in  $O(\log n)$  time.

- **Constraints:**  $1 \leq \text{nums.length} \leq 1000$ ;  $-2^{31} \leq \text{nums}[i] \leq 2^{31} - 1$ ; adjacent elements are guaranteed distinct ( $\text{nums}[i] \neq \text{nums}[i+1]$ ).
- **Clarifying assumptions:** the array is **not** sorted; a single-element array's only element counts as a peak; any valid peak index is accepted, not necessarily the global maximum.
- **Why it's tricky:** the array isn't sorted, so it's not obvious binary search even applies — the trick is realizing the *slope direction* at `mid`, not the value itself, is what's monotonic and guides the search.

### Example.

```
Input:  nums = [1, 2, 3, 1]
Output: 2      (nums[2] = 3 is a peak)
```

**Intuition (diagram).** Look at the slope at `mid`. If it's rising (`nums[mid] < nums[mid+1]`), a peak is guaranteed to exist somewhere to the right — the slope must eventually turn back down because of the `-infinity` boundary. If it's falling (or flat), a peak lies at `mid` or somewhere to the left. "Which way is the slope pointing" is monotonic enough to binary search on, even though the array itself isn't sorted.

```
idx: 0  1  2  3
val: 1  2  3  1
      mid=1: nums[1]=2 < nums[2]=3 -> rising -> peak is strictly to the right of mid

idx: 0  1  2  3
val: 1  2  3  1
      mid=2: nums[2]=3 > nums[3]=1 -> falling -> peak is at mid or to the left
```

## Approach (step by step)

- **Key idea / invariant:** maintain `lo..hi` as a range that is guaranteed to still contain a peak. Comparing `nums[mid]` to its right neighbor tells us which half keeps that guarantee — the loop shrinks the range until `lo == hi`, which must be a peak.
- Set `lo = 0`, `hi = nums.length - 1`.
- While `lo < hi`: compute `mid = lo + (hi - lo) / 2`.
- If `nums[mid] < nums[mid + 1]`, the slope is rising, so a peak exists somewhere in `mid+1 .. hi` — set `lo = mid + 1`.
- Otherwise the slope is falling (or `mid` itself is the top), so a peak exists in `lo .. mid` — set `hi = mid`.
- When `lo == hi`, that index is a peak; return it.
- **Worked trace** (`nums = [1,2,3,1]`): `lo=0, hi=3 -> mid=1, nums[1]=2 < nums[2]=3 -> lo=2 -> mid=2, nums[2]=3 > nums[3]=1 -> hi=2 -> lo == hi == 2 -> return 2`.

## Brute force

Compare every element to both neighbors.

```
public int findPeakBrute(int[] nums) {
    int n = nums.length;
    for (int i = 0; i < n; i++) {
        boolean leftOk = (i == 0) || nums[i - 1] < nums[i];
        boolean rightOk = (i == n - 1) || nums[i] > nums[i + 1];
        if (leftOk && rightOk) return i;
    }
    return -1;
}
```



## Optimized (binary search on slope direction)

```
public int findPeakElement(int[] nums) {
    int lo = 0, hi = nums.length - 1;
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;           // safe midpoint
        if (nums[mid] < nums[mid + 1]) {       // rising slope: peak is to the right
            lo = mid + 1;
        } else {                               // falling slope: peak is at mid or left
            hi = mid;
        }
    }
    return lo;                                // lo == hi lands on a peak
}
```

| Version       | Time        | Space  |
|---------------|-------------|--------|
| Brute force   | $O(n)$      | $O(1)$ |
| Binary search | $O(\log n)$ | $O(1)$ |

## 1.4 Single Element in a Sorted Array (LC 540)

**Problem.** Given a sorted array `nums` in which every element appears exactly twice except for one element that appears exactly once, find and return that single element in  $O(\log n)$  time and  $O(1)$  space.

- **Constraints:**  $1 \leq \text{nums.length} \leq 3 \cdot 10^4$  (length is always odd);  $0 \leq \text{nums}[i] \leq 10^5$ ; the array is sorted ascending.
- **Clarifying assumptions:** exactly one element is unpaired, and every other value appears exactly twice (not three-or-more times); the array length is always odd.
- **Why it's tricky:** the array isn't sorted by "the thing we're searching for" in the usual sense — the values themselves don't tell you which half to go to. The insight is to binary search on **index parity** (whether pairs are still aligned to even indices), which is what actually flips exactly once.

**Example.**

Input: `nums = [1,1,2,2,3,3,4,8,8]`  
Output: 4

**Intuition (diagram).** Before the single element, pairs line up so the first copy of each pair sits at an **even** index (`nums[even] == nums[even+1]`). After the single element, every pair shifts by one slot, so the first copy now sits at an **odd** index instead. Binary search on whether the even-alignment still holds at `mid` (rounded down to even) — that alignment flips exactly once, right at the single element.

idx: 0 1 2 3 4 5 6 7 8  
val: 1 1 2 2 3 3 4 8 8

pairs before: (0,1) (2,3) (4,5)      single -> 6      pairs after: (7,8)  
                 even-index rule holds      |      even-index rule breaks here

## Approach (step by step)

- **Key idea / invariant:** as long as we're left of the single element, every pair's first copy sits at an even index, so `nums[evenIdx] == nums[evenIdx + 1]`. Once we pass the single element, that alignment is permanently broken. This "still aligned?" boolean is monotonic, so we binary search for where it flips.
- Set `lo = 0`, `hi = nums.length - 1`.
- While `lo < hi`: compute `mid = lo + (hi - lo) / 2`, then force it down to the nearest even index (`if (mid % 2 == 1) mid--`).
- If `nums[mid] == nums[mid + 1]`, the pairing is still intact at `mid`, so the single element must be further right — set `lo = mid + 2` (skip the whole intact pair).
- Otherwise the pairing is already broken by `mid`, so the single element is at `mid` or to its left — set `hi = mid`.
- When `lo == hi`, `nums[lo]` is the single element.
- **Worked trace** (`nums = [1,1,2,2,3,3,4,8,8]`): `lo=0`, `hi=8` -> `mid=4` (even) -> `nums[4]=3 == nums[5]=3` -> `lo=6` -> `mid=7` -> round down to `6` -> `nums[6]=4 != nums[7]=8` -> `hi=6` -> `lo == hi == 6` -> return `nums[6] = 4`.

## Brute force

Scan pairs two at a time until the pairing breaks.

```
public int singleNonDuplicateBrute(int[] nums) {
    for (int i = 0; i < nums.length; i += 2) {
        if (i + 1 == nums.length || nums[i] != nums[i + 1]) {
            return nums[i];           // pairing broke here
        }
    }
    return -1;                       // unreachable for valid input
}
```

## Optimized (binary search on pair alignment)

```
public int singleNonDuplicate(int[] nums) {
    int lo = 0, hi = nums.length - 1;
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;           // safe midpoint
        if (mid % 2 == 1) mid--;               // force mid to land on an even index
        if (nums[mid] == nums[mid + 1]) {
            lo = mid + 2;                       // pair intact, single is further right
        } else {
            hi = mid;                           // pairing already broken, single is here
        }
    }
    return nums[lo];
}
```

| Version       | Time        | Space  |
|---------------|-------------|--------|
| Brute force   | $O(n)$      | $O(1)$ |
| Binary search | $O(\log n)$ | $O(1)$ |

## 1.5 Capacity To Ship Packages Within D Days (LC 1011)

**Problem.** Given an array `weights` where `weights[i]` is the weight of the `i`-th package (packages must ship in this exact order) and an integer `days`, find the minimum ship capacity so that all packages can be shipped within `days` days. Each day you load consecutive packages onto the ship, in order, without letting the running load exceed the capacity, then start a fresh day.

- **Constraints:** `1 <= days <= weights.length <= 5*10^4`; `1 <= weights[i] <= 500`.
- **Clarifying assumptions:** packages can't be split or reordered; each package must fit on some day by itself, so the capacity can never be less than the heaviest single package; a day can hold zero packages only if there are no packages left.
- **Why it's tricky:** it doesn't look like a search problem at all — there's no sorted array of candidates to index into. The insight is that the *answer itself* (the capacity) lives on a number line where "is this capacity good enough?" flips from no to yes exactly once, so you can binary search that line directly.

### Example.

```
Input: weights = [1,2,3,4,5,6,7,8,9,10], days = 5
Output: 15
```

**Intuition (diagram).** This is **binary search on the answer space**: the thing we binary search over is a *capacity*, not an array index. Define `feasible(C)` = "can we ship everything within `days` days if the capacity is `C`?". If `C` works, any bigger `C` also works (more room never hurts) — that's the monotonic yes/no test. So search capacities from `max(weights)` (must fit the single heaviest package) up to `sum(weights)` (ship everything in one day), and for each candidate greedily count how many days it takes.

|              |    |    |                                                  |                                             |
|--------------|----|----|--------------------------------------------------|---------------------------------------------|
| capacity:    | 10 | 14 | 15                                               | 20                                          |
| days needed: | 6  | 5  | 5                                                | 4                                           |
|              |    |    | ^^ smallest capacity with days needed <= 5 is 15 |                                             |
| feasible(C): | no | no | yes                                              | yes ... (flips from no to yes exactly once) |

### Approach (step by step)

- **Key idea / invariant:** `daysNeeded(cap)` is a monotonic non-increasing function of `cap` — a bigger capacity never needs more days. So `feasible(cap) = daysNeeded(cap) <= days` is monotonic non-decreasing in `cap` (once true, stays true), which is exactly the shape binary search needs. `lo..hi` always bounds the smallest feasible capacity.
- Set `lo = max(weights)` (any smaller capacity can't even hold the heaviest package) and `hi = sum(weights)` (shipping everything in one day always works).
- While `lo < hi`: compute `mid = lo + (hi - lo) / 2` — a candidate capacity.
- Run the greedy simulation `daysNeeded(mid)`: walk the weights, adding each to the current day's load; whenever adding one would exceed `mid`, start a new day.
- If `daysNeeded(mid) <= days`, `mid` is feasible — it might still be larger than necessary, so try to shrink it: `hi = mid`.

- Otherwise `mid` is too small (needs too many days) — raise the floor: `lo = mid + 1`.
- When `lo == hi`, that's the minimum feasible capacity.
- **Worked trace** (`weights=[1..10]`, `days=5`): `lo=10`, `hi=55` -> `mid=32`, `daysNeeded(32)` is small, feasible, `hi=32` -> ... narrowing continues -> eventually `mid=15`, `daysNeeded(15)=5`  $\leq 5$ , feasible, `hi=15`; `mid=14` (a lower candidate along the way), `daysNeeded(14)=6`  $> 5$ , infeasible, `lo=15` -> `lo == hi == 15`.

## Brute force

Try every capacity starting at `max(weights)`, simulate the shipping schedule.

```
public int shipWithinDaysBrute(int[] weights, int days) {
    int maxWeight = 0, total = 0;
    for (int w : weights) { maxWeight = Math.max(maxWeight, w); total += w; }
    for (int cap = maxWeight; cap <= total; cap++) {
        if (daysNeeded(weights, cap) <= days) return cap;    // first feasible capacity
    }
    return total;
}

private int daysNeeded(int[] weights, int cap) {
    int days = 1, load = 0;
    for (int w : weights) {
        if (load + w > cap) { days++; load = 0; }            // start a new day
        load += w;
    }
    return days;
}
```

## Optimized (binary search on answer space)

Binary search the capacity directly; the feasibility test is monotonic.

```
public int shipWithinDays(int[] weights, int days) {
    int lo = 0, hi = 0;
    for (int w : weights) { lo = Math.max(lo, w); hi += w; } // lo = heaviest package, hi =
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;                // safe midpoint: candidate capacity
        if (daysNeeded(weights, mid) <= days) {
            hi = mid;                                // capacity works, try to shrink it
        } else {
            lo = mid + 1;                            // needs more days, capacity too small
        }
    }
    return lo;
}

private int daysNeeded(int[] weights, int cap) {
    int days = 1, load = 0;
    for (int w : weights) {
        if (load + w > cap) { days++; load = 0; }            // start a new day
        load += w;
    }
    return days;
}
```

| Version                 | Time                          | Space  |
|-------------------------|-------------------------------|--------|
| Brute force             | $O(n \cdot \text{maxSum})$    | $O(1)$ |
| Binary search on answer | $O(n \cdot \log(\text{sum}))$ | $O(1)$ |

## 1.6 Split Array Largest Sum (LC 410)

**Problem.** Given an integer array `nums` and an integer `m`, split `nums` into `m` non-empty **contiguous** subarrays so that the largest sum among the `m` subarrays is as small as possible. Return that minimized largest sum.

- **Constraints:** `1 <= nums.length <= 1000`; `0 <= nums[i] <= 10^6`; `1 <= m <= min(50, nums.length)`.
- **Clarifying assumptions:** every subarray must be non-empty and contiguous (no reordering, no skipping elements); `m` never exceeds the array length, so a valid split always exists.
- **Why it's tricky:** it looks like a partition/DP problem (and it does have an  $O(n^2 \cdot m)$  DP solution), but there's a much faster reframing — binary search the *value* of the answer itself, using a cheap greedy check to test feasibility of each candidate.

**Example.**

```
Input:  nums = [7,2,5,10,8], m = 2
Output: 18      (split as [7,2,5] | [10,8]: sums 14 and 18)
```

**Intuition (diagram).** Same shape as 1.5: binary search over the *answer* — a candidate "max sum allowed per part," call it `X`. The monotonic feasibility test is: **greedily** pack numbers into the current part until adding the next one would exceed `X`, then start a new part — how many parts does that take? If it takes `<= m` parts, `X` is feasible (and so is every larger `X`, since a bigger cap only merges parts); if it takes more than `m` parts, `X` is too small. Binary search the smallest feasible `X` between `max(nums)` (a single huge element forces at least this much) and `sum(nums)` (one part holds everything).

```
nums:  7  2  5  10  8      m = 2

try X = 18: greedy -> [7,2,5]=14 | [10,8]=18      -> 2 parts <= m   FEASIBLE
try X = 17: greedy -> [7,2,5]=14 | [10]=10 | [8]=8 -> 3 parts  > m   TOO SMALL

answer = smallest feasible X = 18
```

### Approach (step by step)

- **Key idea / invariant:** `partsNeeded(cap)` (greedily pack until the next element would overflow `cap`, then start a new part) is monotonic non-increasing in `cap` — a bigger cap only ever merges parts, never splits them further. So `feasible(cap) = partsNeeded(cap) <= m` flips from false to true exactly once as `cap` grows, which is what makes binary search valid here.
- Set `lo = max(nums)` (the largest single element is a hard floor — no cap smaller than it could ever hold that element) and `hi = sum(nums)` (one giant part always works).
- While `lo < hi`: compute `mid = lo + (hi - lo) / 2` — a candidate cap.

- Run `partsNeeded(mid)` : walk `nums` left to right, accumulating a running sum; the moment adding the next element would exceed `mid` , close the current part and start a new one with that element.
- If `partsNeeded(mid) <= m` , `mid` is feasible — try a smaller cap: `hi = mid` .
- Otherwise `mid` is too small (too many parts needed) — raise the floor: `lo = mid + 1` .
- When `lo == hi` , that's the minimized largest sum.
- **Worked trace** (`nums=[7,2,5,10,8]` , `m=2`): `lo=10`, `hi=32` -> `mid=21` , `partsNeeded(21)` : `[7,2,5]=14` , `+10=24>21` so new part `[10,8]=18` -> 2 parts `<= 2`, feasible, `hi=21` -> `mid=15` , `partsNeeded(15)` : `[7,2,5]=14` , `+10>15` new part `[10]=10` , `+8>15` new part `[8]` -> 3 parts, infeasible, `lo=16` -> `mid=18` , `partsNeeded(18)` : `[7,2,5]=14` , `+10>18` new part `[10,8]=18` -> 2 parts, feasible, `hi=18` -> `mid=17` , `partsNeeded(17)` needs 3 parts, infeasible, `lo=18` -> `lo == hi == 18` .

## Brute force

Try every candidate max-sum from `max(nums)` to `sum(nums)` , check feasibility.

```
public int splitArrayBrute(int[] nums, int m) {
    int maxVal = 0, total = 0;
    for (int n : nums) { maxVal = Math.max(maxVal, n); total += n; }
    for (int x = maxVal; x <= total; x++) {
        if (partsNeeded(nums, x) <= m) return x;    // first feasible max-sum
    }
    return total;
}

private int partsNeeded(int[] nums, int maxSum) {
    int parts = 1, running = 0;
    for (int n : nums) {
        if (running + n > maxSum) { parts++; running = 0; }
        running += n;
    }
    return parts;
}
```

## Optimized (binary search on answer + greedy feasibility check)

```
public int splitArray(int[] nums, int m) {
    int lo = 0, hi = 0;
    for (int n : nums) { lo = Math.max(lo, n); hi += n; } // lo = biggest single element,
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2; // safe midpoint: candidate max-sum-per-part
        if (partsNeeded(nums, mid) <= m) { // fits in <= m parts, try a smaller cap
            hi = mid;
        } else { // needs too many parts, cap too small
            lo = mid + 1;
        }
    }
    return lo;
}

private int partsNeeded(int[] nums, int maxSum) {
    int parts = 1, running = 0;
    for (int n : nums) {
        if (running + n > maxSum) { // this element would overflow the current part
            parts++; // start a new part
            running = 0;
        }
        running += n;
    }
    return parts;
}
```

| Version                 | Time                          | Space  |
|-------------------------|-------------------------------|--------|
| Brute force             | $O(n \cdot \text{sum})$       | $O(1)$ |
| Binary search on answer | $O(n \cdot \log(\text{sum}))$ | $O(1)$ |

## 1.7 Median of Two Sorted Arrays (LC 4)

**Problem.** Given two sorted arrays **A** (size **m**) and **B** (size **n**), return the median of the array formed by merging them, without actually merging them, in  $O(\log(\min(m, n)))$  time.

- **Constraints:**  $0 \leq m, n$ ;  $1 \leq m + n$  (the combined array is never empty);  $-10^6 \leq A[i], B[i] \leq 10^6$ ; both arrays are sorted ascending.
- **Clarifying assumptions:** either array (but not both) may be empty; duplicates across or within the arrays are allowed and don't change the approach.
- **Why it's tricky:** the  $O(\log(m+n))$  time bound rules out merging ( $O(m+n)$ ) and even rules out a straightforward binary search for the *value* of the median. The real target of the search is a **partition point** — a way of cutting both arrays so the left pieces together hold exactly the left half of the combined data — not a value or an array index in the usual sense.

**Example.**

Input: **A** = [1, 3], **B** = [2]  
Output: 2.0

**Intuition (diagram).** Don't merge the arrays. Binary search for a **partition** on the smaller array (say  $A$ ): pick a cut  $i$  in  $A$ , which forces a matching cut  $j = (m+n+1)/2 - i$  in  $B$ , so together the left side holds exactly half the combined elements (rounding up for odd totals). The partition is valid exactly when  $A[i-1] \leq B[j]$  **and**  $B[j-1] \leq A[i]$  — everything on the left is  $\leq$  everything on the right. If  $A[i-1] > B[j]$ ,  $A$ 's left slice reaches too far right, so move the cut left; otherwise move it right. "Does the left reach too far right?" is monotonic in  $i$ , so it's a binary search over cut positions, not over values.

```
A:  1  3      (m = 2)      cut i = 1 -> A_left = [1]      A_right = [3]
B:  2      (n = 1)      cut j = 1 -> B_left = [2]      B_right = []

combined left = {1, 2}      combined right = {3}

check: A[i-1] = 1 <= B[j] = +inf   and   B[j-1] = 2 <= A[i] = 3   -> valid partition
total length 3 (odd) -> median = max(left) = max(1, 2) = 2
```

## Approach (step by step)

- **The partition invariant, carefully.** Imagine the fully merged, sorted array of all  $m + n$  elements split into a "left half" and a "right half" at the median. We never build that merged array; instead we find, directly, a cut  $i$  in  $A$  and a cut  $j$  in  $B$  such that:
- **Size invariant:**  $i + j$  always equals  $(m + n + 1) / 2$  — the number of elements that belong in the combined left half (integer division rounds up, so for odd totals the left half has one extra element, which becomes the median by itself). Because  $j$  is *derived* from  $i$  ( $j = (m+n+1)/2 - i$ ), we only ever need to binary search  $i$ ;  $j$  is forced along with it.
- **Order invariant:** every element in  $A\_left \cup B\_left$  must be  $\leq$  every element in  $A\_right \cup B\_right$ . Since  $A\_left$  is already  $\leq A\_right$  (same array, sorted) and likewise for  $B$ , the only cross-array checks needed are  $A[i-1] \leq B[j]$  (last of  $A\_left$  vs first of  $B\_right$ ) and  $B[j-1] \leq A[i]$  (last of  $B\_left$  vs first of  $A\_right$ ). If both hold, the two "left" pieces interleave into a valid combined left half with no reordering needed.
- Treat a cut at the very start/end of an array as  $-\infty / +\infty$  sentinels ( $aLeft$ ,  $aRight$ ,  $bLeft$ ,  $bRight$  in the code) so the boundary cuts ( $i = 0$ ,  $i = m$ , etc.) don't need special-case branches.
- **Why binary search  $i$  works:** as  $i$  increases,  $A\_left$  only grows (monotonically admits more/larger  $A$  elements) and  $j$  shrinks to compensate, so  $A[i-1]$  is non-decreasing and  $B[j]$  is non-increasing in  $i$ . That means " $A[i-1] > B[j]$ " (left reaches too far right) is monotonic in  $i$  — once true it stays true as  $i$  grows — so we can binary search for the boundary between "too far right" and "not far enough."
- Always search the **smaller** array as  $A$  (swap if needed) so  $j$  never goes negative and the search space is as small as possible.
- Set  $lo = 0$ ,  $hi = m$  ( $i$  ranges over "0 elements of  $A$  on the left" to "all of  $A$  on the left").
- While  $lo \leq hi$ : compute  $i = lo + (hi - lo) / 2$ , then  $j = (m+n+1)/2 - i$ .
- Read the four boundary values ( $aLeft$ ,  $aRight$ ,  $bLeft$ ,  $bRight$ ), substituting  $\pm\infty$  sentinels when a cut sits at an array's edge.
- If  $aLeft \leq bRight$  &&  $bLeft \leq aRight$ , the partition is valid: for odd total length the median is  $\max(aLeft, bLeft)$ ; for even length it's the average of  $\max(aLeft, bLeft)$  and  $\min(aRight, bRight)$ .
- If  $aLeft > bRight$ ,  $A$ 's left slice reached too far right — shrink it:  $hi = i - 1$ .
- Otherwise  $A$ 's left slice isn't big enough — grow it:  $lo = i + 1$ .



- **Worked trace** ( $A = [1, 3]$ ,  $B = [2]$ ,  $m = 2$ ,  $n = 1$ ): smaller array is already  $A$ .  $lo = 0$ ,  $hi = 2$ .
- $i = 1$  ( $lo + (hi-lo)/2 = 0 + 1$ ),  $j = (2+1+1)/2 - 1 = 2 - 1 = 1$ .
- $aLeft = A[0] = 1$ ,  $aRight = A[1] = 3$ ,  $bLeft = B[0] = 2$ ,  $bRight = +infinity$  (since  $j == n$ ).
- Check:  $aLeft=1 \leq bRight=+inf$  true,  $bLeft=2 \leq aRight=3$  true  $\rightarrow$  valid partition.
- Total length 3 is odd  $\rightarrow$  median  $= \max(aLeft, bLeft) = \max(1, 2) = 2$ . Matches the expected output 2.0.

## Brute force

Merge both arrays, then read off the middle.

```
public double findMedianSortedArraysBrute(int[] a, int[] b) {
    int[] merged = new int[a.length + b.length];
    int i = 0, j = 0, k = 0;
    while (i < a.length && j < b.length) merged[k++] = (a[i] <= b[j]) ? a[i++] : b[j++];
    while (i < a.length) merged[k++] = a[i++];
    while (j < b.length) merged[k++] = b[j++];

    int n = merged.length, mid = n / 2;
    if (n % 2 == 1) return merged[mid];
    return (merged[mid - 1] + merged[mid]) / 2.0;
}
```

## Optimized (binary search on partition, smaller array)

```
public double findMedianSortedArrays(int[] a, int[] b) {
    if (a.length > b.length) return findMedianSortedArrays(b, a); // always search the smaller array
    int m = a.length, n = b.length;
    int lo = 0, hi = m; // number of elements of A placed on the left
    while (lo <= hi) {
        int i = lo + (hi - lo) / 2; // safe midpoint: cut position in A
        int j = (m + n + 1) / 2 - i; // matching cut in B so left half has (m+n+1)/2 elements

        int aLeft = (i == 0) ? Integer.MIN_VALUE : a[i - 1];
        int aRight = (i == m) ? Integer.MAX_VALUE : a[i];
        int bLeft = (j == 0) ? Integer.MIN_VALUE : b[j - 1];
        int bRight = (j == n) ? Integer.MAX_VALUE : b[j];

        if (aLeft <= bRight && bLeft <= aRight) { // valid partition found
            if ((m + n) % 2 == 1) {
                return Math.max(aLeft, bLeft);
            }
            return (Math.max(aLeft, bLeft) + Math.min(aRight, bRight)) / 2.0;
        } else if (aLeft > bRight) {
            hi = i - 1; // A's left slice reaches too far right, cut earlier
        } else {
            lo = i + 1; // A's left slice doesn't reach far enough, cut later
        }
    }
    throw new IllegalArgumentException("input arrays are not sorted");
}
```

| Version                    | Time                  | Space      |
|----------------------------|-----------------------|------------|
| Brute force (merge)        | $O(m + n)$            | $O(m + n)$ |
| Binary search on partition | $O(\log(\min(m, n)))$ | $O(1)$     |

---

## 2. Tries (Prefix Trees)

---

**The core idea.** A trie is a tree built for strings: each **edge** represents one character, not each node. Words that share a prefix share the same path from the root, so "cat" and "car" walk the same two edges (c -> a) before splitting. Each node holds up to 26 children (one slot per lowercase letter) plus an `isEnd` flag marking "a real word ends here." Walking down the tree one character at a time is how you check prefixes, insert words, and search — all in time proportional to the word's length, not the size of the dictionary.

**Reach for it when:** you see **prefix**, **startsWith**, **dictionary / word list**, or **autocomplete** in the problem — anything where many strings share leading characters and you need fast prefix lookups.

Reusable node definition used throughout this section:

```
class TrieNode {
    TrieNode[] children = new TrieNode[26]; // index = letter - 'a'
    boolean isEnd; // true if a word ends at this node
}
```

### 2.1 Implement Trie (Prefix Tree)

**Problem.** Design a trie (prefix tree) that supports three operations: `insert(word)` — add a word to the trie; `search(word)` — return `true` only if that **exact** word was previously inserted; `startsWith(prefix)` — return `true` if **any** inserted word begins with `prefix`. All three operations must run fast even after many inserts.

- **Given:** a stream of `insert` calls, interleaved with `search` / `startsWith` queries.
- **Return:** a boolean for each `search` / `startsWith` call.

**Constraints** -  $1 \leq \text{word.length}, \text{prefix.length} \leq 2000$ . - `word` and `prefix` consist of lowercase English letters only ( 'a' - 'z' ). - Up to  $3 * 10^4$  total calls across `insert`, `search`, and `startsWith`.

**Clarifying assumptions** - Confirm the alphabet is lowercase `a - z` only — that's what justifies a fixed 26-slot array per node instead of a hash map. - The same word may be inserted more than once; `insert` should stay correct (idempotent) if that happens.

**Why it's tricky:** `search` must distinguish "this path exists in the tree" from "a word actually ends here." After inserting "apple", the prefix "app" exists as a path, but `search("app")` must still return `false` until "app" is inserted as its own word — that's exactly what the `isEnd` flag is for.

**Example.**

```

insert("apple")
search("apple")    -> true
search("app")      -> false    (no word ends exactly at "app")
startsWith("app")  -> true     (the path a-p-p exists)
insert("app")
search("app")      -> true

```

**Intuition (diagram).** Inserting "apple" then "app" builds one shared path; "app" just flips `isEnd` on an existing node instead of creating a new branch.

```

root
|
a
|
p
|
p  <- isEnd=true (after inserting "app")
|
l
|
e  <- isEnd=true (after inserting "apple")

```

## Approach (step by step)

**Node structure.** Every `TrieNode` is just two fields: a fixed-size array `children[26]` (slot `i` holds the child reached by letter `'a' + i`, or `null` if no word uses that branch here) and a boolean `isEnd` (true only if some inserted word stops exactly at this node). The root is an empty `TrieNode` representing "zero characters consumed."

**Key idea / invariant.** At every point during a walk, the current `TrieNode` represents "the set of inserted words that share the prefix consumed so far." Following child `i` extends that prefix by one letter; `isEnd` on the current node tells you whether the prefix consumed so far is itself a complete inserted word.

**insert(word) — numbered walk-through:** 1. Start `node = root` (0 characters consumed). 2. For each character `c` in `word`, compute its slot `i = c - 'a'`. 3. If `node.children[i]` is `null`, create a new `TrieNode` there — this is a new branch. 4. Move `node = node.children[i]` and repeat for the next character. 5. After the last character, set `node.isEnd = true` — a real word terminates here.

**search(word) / startsWith(prefix) — numbered walk-through:** 1. Start `node = root` and walk exactly like `insert`, but never create nodes. 2. If any character's slot is `null`, the path breaks — no inserted word has this prefix, so return `false` immediately. 3. If every character's slot exists, you reach a node after consuming the whole string. 4. `startsWith` returns `true` right here — the path existing is enough. 5. `search` additionally checks `node.isEnd` — the path must exist **and** end exactly there for a real word.

**Worked trace** (after `insert("apple")`, `insert("app")`): - `search("app")` before the second insert: walk `a -> p -> p`, path exists, but `isEnd` is `false` at that node (only "apple" was inserted) -> returns `false`. - After `insert("app")`, the node at `a -> p -> p` gets `isEnd = true` (no new nodes are created — the path already existed) -> `search("app")` now returns `true`. - `startsWith("app")` returns `true` in both cases, since the path `a -> p -> p` exists either way.

```

class Trie {
    private final TrieNode root = new TrieNode();

    public void insert(String word) {
        TrieNode node = root;
        for (char c : word.toCharArray()) {
            int i = c - 'a';
            if (node.children[i] == null) {
                node.children[i] = new TrieNode();
            }
            node = node.children[i];
        }
        node.isEnd = true;
    }

    public boolean search(String word) {
        TrieNode node = walk(word);
        return node != null && node.isEnd;
    }

    public boolean startsWith(String prefix) {
        return walk(prefix) != null;
    }

    // returns the node reached after consuming `s`, or null if the path breaks
    private TrieNode walk(String s) {
        TrieNode node = root;
        for (char c : s.toCharArray()) {
            int i = c - 'a';
            if (node.children[i] == null) return null;
            node = node.children[i];
        }
        return node;
    }
}

```

| Version                         | Time        | Space                          |
|---------------------------------|-------------|--------------------------------|
| Trie (insert/search/startsWith) | O(L) per op | O(N · L) total for all inserts |

## 2.2 Design Add and Search Words Data Structure

**Problem.** Design a data structure that supports `addWord(word)` — add a word — and `search(word)` — return `true` if some previously added word matches `word` exactly, where `word` may contain the wildcard character `.`, which matches **any single letter** at that position.

- **Given:** a stream of `addWord` calls, then `search` queries that may embed `.`.
- **Return:** a boolean per `search` call.

**Constraints** -  $1 \leq \text{word.length} \leq 25$ , lowercase English letters and possibly `.` in `search` queries (never in `addWord`). - Up to  $3 \cdot 10^4$  calls to `addWord`. - Up to  $10^4$  calls to `search`; at most a small number of `.` per query in practice, but a query of all dots is legal (e.g. `"..."`).

**Clarifying assumptions** - Confirm `.` never appears in an added word, only in search queries. - Confirm word length is bounded (small, e.g.  $\leq 25$ ) — this matters because a query of all dots triggers a full fan-

out and the exponential blow-up must stay tractable.

**Why it's tricky:** a single `.` can match multiple different branches at once, so `search` isn't a straight-line walk anymore — it's a DFS that tries every child when it hits a wildcard, and must OR together the results from all branches explored.

### Example.

```
addWord("bad")
addWord("dad")
addWord("mad")
search("pad")  -> false
search("bad")  -> true
search(".ad")  -> true   ('.' matches 'b', 'd', or 'm')
search("b..")  -> true
```

**Intuition (diagram).** A normal letter follows one edge; a `.` means "try every existing edge from this node" — so the search fans out into a DFS instead of a single path.

```
root
|-- b -- a -- d  <- isEnd ("bad")
|-- d -- a -- d  <- isEnd ("dad")
|-- m -- a -- d  <- isEnd ("mad")

search(".ad"): at root, '.' fans out into all three branches (b, d, m);
               each branch then matches 'a' (single edge) and 'd' (check isEnd) normally.
```

### Approach (step by step)

**Node structure.** Same `TrieNode` as 2.1: `children[26]` array plus `isEnd`. Nothing changes about how nodes are built.

**Key idea / invariant.** `addWord` is identical to `insert` in 2.1 — wildcards never appear when adding. The new piece is `search`: define `dfs(node, word, idx) = "can the suffix word[idx..] be matched starting from node?"` The invariant is that `node` always represents the trie position reached after successfully matching `word[0..idx)` along *one specific branch* being explored.

**search(word) — numbered walk-through (recursive DFS):** 1. Call `dfs(root, word, 0)`. 2. Base case — if `node` is `null` (path already broken), return `false`. 3. Base case — if `idx == word.length()` (whole query consumed), return `node.isEnd` (must land exactly on a real word, same as plain trie search). 4. Otherwise look at `c = word.charAt(idx)`: - If `c` is a normal letter: recurse once into `node.children[c - 'a']` with `idx + 1`. Exactly one branch, just like 2.1. - If `c == '.'`: loop over **every** slot in `node.children`; for each non-null child, recurse with `idx + 1`. Return `true` as soon as any branch returns `true` (short-circuit OR); if none do, return `false`. 5. This is exactly DFS with branching factor 26 at every wildcard, and branching factor 1 at every normal letter — the trie's shared prefixes keep the fan-out from re-walking duplicate work across the three inserted words.

**Worked trace** for `search(".ad")` after adding `"bad"`, `"dad"`, `"mad"`: - `idx=0`, `c='.'` at `root`: fan out to children `b`, `d`, `m` (three recursive calls). - Branch `b`: `idx=1`, `c='a'`, single child -> move to node `a` under `b`. - Branch `b`: `idx=2`, `c='d'`, single child -> move to node `d` under `b-a`. - Branch `b`: `idx=3 == length`, check `isEnd` -> `true` -> whole call returns `true` immediately (branches `d` and `m` are never even explored because of short-circuiting).

```

class WordDictionary {
    private final TrieNode root = new TrieNode();

    public void addWord(String word) {
        TrieNode node = root;
        for (char c : word.toCharArray()) {
            int i = c - 'a';
            if (node.children[i] == null) {
                node.children[i] = new TrieNode();
            }
            node = node.children[i];
        }
        node.isEnd = true;
    }

    public boolean search(String word) {
        return dfs(root, word, 0);
    }

    private boolean dfs(TrieNode node, String word, int idx) {
        if (node == null) return false;
        if (idx == word.length()) return node.isEnd;

        char c = word.charAt(idx);
        if (c == '.') {
            for (TrieNode child : node.children) {
                if (dfs(child, word, idx + 1)) return true;
            }
            return false;
        } else {
            return dfs(node.children[c - 'a'], word, idx + 1);
        }
    }
}

```

| Version       | Time                                                        | Space                       |
|---------------|-------------------------------------------------------------|-----------------------------|
| DFS over trie | $O(L)$ exact word; up to $O(26^L)$ worst case with all dots | $O(N \cdot L)$ for the trie |

## 2.3 Replace Words

**Problem.** Given a dictionary of word **roots** and a **sentence** (space-separated words), replace every word in the sentence with the **shortest** root from the dictionary that is a prefix of it. If several roots match, use the shortest; if no root matches, leave the word unchanged. Return the resulting sentence.

- **Given:** `roots: List<String>`, `sentence: String`.
- **Return:** the modified sentence as a single string, words separated by single spaces.

**Constraints** -  $1 \leq \text{roots.length} \leq 1000$ ,  $1 \leq \text{roots}[i].\text{length} \leq 100$  . -  $1 \leq \text{sentence.length} \leq 10^6$  total characters; sentence words are separated by single spaces, lowercase English letters only.  
 - Roots and sentence words consist of lowercase English letters only.

**Clarifying assumptions** - Confirm "root" means prefix match, not substring match — `"a"` matches `"aa"` but not `"ba"` . - Confirm ties on length don't matter (any shortest root is equally valid; there's at

most one shortest root that's a prefix, since two same-length prefixes of the same word must be identical).

**Why it's tricky:** checking every word against every root with `String.startsWith` is  $O(\text{words} * \text{roots} * L)$  — fine for small inputs but wasteful once `roots` is large; a trie lets each sentence word be checked against **all** roots simultaneously in one walk down the tree, stopping the instant it passes a real root node.

### Example.

```
Input:  roots = ["cat", "bat", "rat"], sentence = "the cattle was rattled by the battery"
Output: "the cat was rat by the bat"
```

**Intuition (diagram).** Insert every root into a trie. For each sentence word, walk the trie character by character; the moment you hit a node with `isEnd = true`, you've found the shortest root — stop immediately.

```
root
|-- c -- a -- t  <- isEnd ("cat")
|-- b -- a -- t  <- isEnd ("bat")
|-- r -- a -- t  <- isEnd ("rat")

"cattle": walk c -> a -> t, isEnd hit at "t" -> stop and replace with "cat"
          (the remaining letters "tle" are never even looked at)
```

### Approach (step by step)

**Node structure.** Same `TrieNode` as 2.1 (`children[26]` + `isEnd`); each root word is inserted exactly like `insert` in 2.1.

**Key idea / invariant.** Build the trie once from all roots. For each sentence word, walk the trie one character at a time while building up the prefix consumed so far. The moment the current node has `isEnd == true`, that prefix is a complete root **and**, because it's the first one hit while descending from the root, it is necessarily the *shortest* root that matches — stop and use it immediately.

**shortestRoot(word) — numbered walk-through:** 1. Start `node = trieRoot`, `prefix = ""`. 2. For each character `c` in `word`: compute slot `i = c - 'a'`. 3. If `node.children[i]` is `null`, no root matches this word at all — return the original `word` unchanged. 4. Otherwise append `c` to `prefix` and move `node = node.children[i]`. 5. If `node.isEnd` is now `true`, `prefix` is the shortest matching root — return it immediately (don't keep consuming the rest of `word`). 6. If the loop finishes without ever hitting `isEnd`, no root is a prefix of the whole word — return the original `word` unchanged. 7. Apply this to every word in the sentence and join with spaces.

**Worked trace** for "cattle" with roots "cat", "bat", "rat": - `c`: slot exists, `prefix = "c"`, `isEnd = false`. - `a`: slot exists, `prefix = "ca"`, `isEnd = false`. - `t`: slot exists, `prefix = "cat"`, `isEnd = true` -> return "cat" right away; the remaining letters "tle" are never visited.

### Brute force

For each word, check it against every root's `startsWith`, keep the shortest match.



```

public String replaceWordsBrute(List<String> roots, String sentence) {
    String[] words = sentence.split(" ");
    for (int w = 0; w < words.length; w++) {
        String best = null;
        for (String root : roots) {
            if (words[w].startsWith(root) && (best == null || root.length() < best.length()))
                best = root;
        }
        if (best != null) words[w] = best;
    }
    return String.join(" ", words);
}

```

## Optimized (trie)

Insert all roots once, then walk the trie for each sentence word and stop at the first `isEnd` .

```

public String replaceWords(List<String> roots, String sentence) {
    TrieNode trieRoot = new TrieNode();
    for (String root : roots) {
        TrieNode node = trieRoot;
        for (char c : root.toCharArray()) {
            int i = c - 'a';
            if (node.children[i] == null) node.children[i] = new TrieNode();
            node = node.children[i];
        }
        node.isEnd = true;
    }

    String[] words = sentence.split(" ");
    for (int w = 0; w < words.length; w++) {
        words[w] = shortestRoot(trieRoot, words[w]);
    }
    return String.join(" ", words);
}

private String shortestRoot(TrieNode trieRoot, String word) {
    TrieNode node = trieRoot;
    StringBuilder prefix = new StringBuilder();
    for (char c : word.toCharArray()) {
        int i = c - 'a';
        if (node.children[i] == null) return word; // no root matches, keep original
        prefix.append(c);
        node = node.children[i];
        if (node.isEnd) return prefix.toString(); // shortest root found
    }
    return word;
}

```

| Version     | Time                                                   | Space                       |
|-------------|--------------------------------------------------------|-----------------------------|
| Brute force | $O(\text{words} \cdot \text{roots} \cdot L)$           | $O(1)$ extra                |
| Trie        | $O(N \cdot L)$ build + $O(\text{words} \cdot L)$ query | $O(N \cdot L)$ for the trie |

## 2.4 Search Suggestions System

**Problem.** Given a list of `products` and a string `searchWord`, simulate typing `searchWord` one character at a time. After each character is typed, return up to **3** products from `products` that share the prefix typed so far, chosen as the **lexicographically smallest** such products (fewer than 3 if fewer match).

- **Given:** `products: String[], searchWord: String`.
- **Return:** `List<List<String>>` — one list of up to 3 suggestions per prefix length.

**Constraints** -  $1 \leq \text{products.length} \leq 1000$ ,  $1 \leq \text{products}[i].\text{length} \leq 3000$ . -  $1 \leq \text{searchWord.length} \leq 1000$ . - All strings consist of lowercase English letters; sum of  $\text{products}[i].\text{length} \leq 2 * 10^4$ . - `products` may contain duplicates in general LeetCode statements, but assume distinct entries here unless told otherwise.

**Clarifying assumptions** - Once no product matches the prefix typed so far, confirm that no *longer* prefix can match either (adding characters only narrows the match set) — so once a prefix fails, every subsequent, longer prefix is guaranteed to return an empty list too.

**Why it's tricky:** naively re-filtering all products for every one of `searchWord`'s prefixes is easy but repeats work; the efficient version needs a trie where each node *pre-computes and caches* its own up-to-3 suggestion list at insert time, which only works cleanly if products are inserted in **sorted order** first.

**Example.**

```
Input: products = ["mobile","mouse","moneypot","monitor","mousepad"], searchWord = "mouse"
Output: [
  ["mobile","moneypot","monitor"], // prefix "m"
  ["mobile","moneypot","monitor"], // prefix "mo"
  ["mouse","mousepad"],           // prefix "mou"
  ["mouse","mousepad"],           // prefix "mous"
  ["mouse","mousepad"],           // prefix "mouse"
]
```

**Intuition (diagram).** Sort products first, then build a trie where each node also stores (up to 3) product indices for words passing through it in sorted order. Typing a prefix is just walking the trie and reading off the stored suggestions.

```
sorted: mobile, moneypot, monitor, mouse, mousepad

root -- m -- o -- b -- i -- l -- e
      | suggestions@"mob": [mobile, moneypot, monitor]
      |-- n -- ...
      | suggestions@"mon": [moneypot, monitor]
      |-- u -- s -- e
          | suggestions@"mou": [mouse, mousepad]
          |-- p -- a -- d
```

**Approach (step by step)**

**Node structure.** `SuggestNode` extends the basic trie node: `children[26]` as usual, plus a `List<String>` `suggestions` capped at 3 entries. There's no separate `isEnd` flag here — the

`suggestions` list itself doubles as "which words pass through this node."

**Key idea / invariant.** If products are inserted into the trie **in sorted order**, then at every node, the first (up to) 3 products that ever get inserted through that node are automatically the 3 lexicographically smallest ones that share that prefix — no sorting or comparison needed at query time, just "keep the first 3 you see, ignore the rest."

**Build phase — numbered walk-through:** 1. Sort `products` lexicographically once, up front. 2. For each product (now in sorted order), walk the trie character by character, creating child nodes as needed (same as `insert` in 2.1). 3. At **every** node visited along the way (not just the last one), if that node's `suggestions` list has fewer than 3 entries, append the current product to it. 4. Because products are processed in sorted order, each node's list fills up with its 3 smallest matching products and then simply stops growing.

**Query phase — numbered walk-through:** 1. Start `node = root`, `broken = false`. 2. For each character `c` in `searchWord`: if not yet `broken` and `node.children[c-'a']` exists, move `node` there and append `node.suggestions` to the result. 3. Otherwise (child missing, or already broken), append an empty list and set `broken = true` — once a prefix has zero matches, every longer prefix must also have zero matches, so there's no need to keep checking children. 4. Return the collected list of up-to-3-item lists, one per character typed.

**Worked trace** for `searchWord = "mou"` (sorted products: mobile, moneypot, monitor, mouse, mousepad): - `m`: node exists, `suggestions = [mobile, moneypot, monitor]` (first 3 seen). - `o`: node exists, `suggestions` still `[mobile, moneypot, monitor]` (same 3, "mo" is a shorter prefix of the same words plus more). - `u`: node exists, `suggestions = [mouse, mousepad]` (only 2 products share "mou").

## Brute force

Sort once; for each prefix length, linearly scan for up to 3 matches (or binary-search the sorted list for the insertion point of the prefix).

```
public List<List<String>> suggestedProductsBrute(String[] products, String searchWord) {
    Arrays.sort(products);
    List<List<String>> result = new ArrayList<>();
    StringBuilder prefix = new StringBuilder();
    for (char c : searchWord.toCharArray()) {
        prefix.append(c);
        List<String> matches = new ArrayList<>();
        for (String p : products) {
            if (p.startsWith(prefix.toString())) {
                matches.add(p);
                if (matches.size() == 3) break; // already sorted, first 3 are smallest
            }
        }
        result.add(matches);
    }
    return result;
}
```

## Optimized (trie with sorted children)

Insert products in sorted order so each node's up-to-3 suggestion list fills in lexicographic order automatically; then just walk the trie once per prefix.

```

class SuggestNode {
    SuggestNode[] children = new SuggestNode[26];
    List<String> suggestions = new ArrayList<>(); // capped at 3, sorted order
}

public List<List<String>> suggestedProducts(String[] products, String searchWord) {
    Arrays.sort(products); // insert in sorted order
    SuggestNode root = new SuggestNode();
    for (String product : products) {
        SuggestNode node = root;
        for (char c : product.toCharArray()) {
            int i = c - 'a';
            if (node.children[i] == null) node.children[i] = new SuggestNode();
            node = node.children[i];
            if (node.suggestions.size() < 3) {
                node.suggestions.add(product); // first 3 seen = 3 smallest
            }
        }
    }

    List<List<String>> result = new ArrayList<>();
    SuggestNode node = root;
    boolean broken = false;
    for (char c : searchWord.toCharArray()) {
        int i = c - 'a';
        if (!broken && node.children[i] != null) {
            node = node.children[i];
            result.add(node.suggestions);
        } else {
            broken = true; // no more matches possible
            result.add(new ArrayList<>());
        }
    }
    return result;
}

```

| Version                   | Time                                                     | Space                       |
|---------------------------|----------------------------------------------------------|-----------------------------|
| Sort + scan (brute force) | $O(N \log N + N \cdot L)$                                | $O(1)$ extra                |
| Trie with sorted children | $O(N \log N + N \cdot L)$ build, $O(L)$ per prefix query | $O(N \cdot L)$ for the trie |

## 2.5 Word Search II

**Problem.** Given an  $m \times n$  grid `board` of lowercase letters and a list of `words`, return every word (no duplicates in the output) that can be traced as a path of **horizontally or vertically adjacent** cells, where **no cell is reused within a single word's path** (a fresh set of "used" cells for each candidate word/path).

- **Given:** `board: char[m][n]`, `words: String[]`.
- **Return:** `List<String>` of the words found on the board, each listed once.

**Constraints** -  $1 \leq m, n \leq 12$ ,  $1 \leq \text{words.length} \leq 3 \cdot 10^4$ . -  $1 \leq \text{words}[i].\text{length} \leq 10$ , all lowercase English letters; words may repeat letters. - Board cells are lowercase English letters.

**Clarifying assumptions** - Confirm diagonal moves are not allowed — only up/down/left/right. - Confirm the same word can only be reported once even if it can be formed via multiple distinct paths on the board.

**Why it's tricky:** running a separate DFS per word (as in Word Search I) repeats the same board exploration once per word, even when many words share a prefix. The efficient solution builds **one** trie from all the words, then runs a **single** DFS sweep over the board, letting the trie itself decide which neighbors are worth visiting — the moment a path leaves every word's prefix set, that branch is pruned instead of explored further.

**Example.**

```
Input: board = [['o','a','a','n'],
               ['e','t','a','e'],
               ['i','h','k','r'],
               ['i','f','l','v']]
        words = ["oath","pea","eat","rain"]
Output: ["eat","oath"]
```

**Intuition (diagram).** Build one trie from all the words, then do a **single** DFS sweep over the board. At each cell, only step into a trie child that exists — the trie prunes whole branches of the search the instant no word matches that prefix, instead of restarting the board search from scratch for every word.

```
words: "oath", "oat", "eat"
```

```
root -- o -- a -- t -- h
          |   |   |
          |   |   isEnd ("oath")
          |   isEnd ("oat")
          |
          (no isEnd here)
root -- e -- a -- t
          |
          isEnd ("eat")
```

DFS at board cell 'o': only follow neighbors matching trie child 'a', never 'e' or others

## Approach (step by step)

**Node structure.** Same `children[26]` array as 2.1, but instead of a plain boolean `isEnd`, this trie stashes `String word` at the node where a word ends (`null` otherwise). That's a small optimization: when the DFS lands on a node with `word != null`, it can add the *found word* directly without needing to reconstruct it from the path taken.

**Key idea / invariant.** Build one trie containing all of `words`. Then run a DFS from **every** board cell, but only ever step into a neighbor whose letter matches an existing child of the **current trie node** — never the current board cell's neighbors blindly. This means the DFS and the trie walk advance in lockstep: `node` always represents "the trie position reached after the path walked on the board so far," so the instant no word shares that prefix, `node.children[...]` is `null` and the branch is abandoned immediately.

**Numbered walk-through:** 1. **Build the trie.** Insert every word from `words` into one shared trie (same char-by-char walk as `insert` in 2.1), storing the full word string at its terminal node instead of a boolean

flag. 2. **Sweep every cell.** For each board cell  $(r, c)$ , call `dfs(r, c, root)` — every cell is a potential starting point for some word. 3. **Inside `dfs(r, c, node)`** : bail out immediately (return) if  $(r, c)$  is out-of-bounds, already visited in this path ( '#' marker), or if `node` has no child for `board[r][c]` — this last check is the trie-driven pruning that makes the whole approach fast. 4. Otherwise move to `next = node.children[board[r][c] - 'a']`. If `next.word != null`, a full word has just been completed — add it to the results and null out `next.word` so it can't be added twice. 5. Mark the current cell visited (`board[r][c] = '#'`), recurse into all four neighbors passing `next` as the new trie node, then restore the cell's original letter (backtrack) so other paths can still use it. 6. Because every started path shares the same trie, cells that begin a prefix common to several words (e.g. "oat" and "oath") are explored **once** per board position, and paths that match no word's prefix at all are cut off after just one wasted step instead of being explored to full word length.

**Worked trace** on `board = [[o,a,a,n],[e,t,a,e],[i,h,k,r],[i,f,l,v]]` for word "oath" (trie holds "oath", "pea", "eat", "rain"): - Start `dfs(0,0,root)`: `board[0][0] = 'o'`, `root.children['o']` exists -> move to node `o`, mark  $(0,0)$  visited. - Neighbor  $(1,0) = 'e'$ : node `o` has no child `'e'` (only `'a'` does, from "oath") -> pruned instantly, no further recursion down that path. - Neighbor  $(0,1) = 'a'$ : node `o` has child `'a'` -> move to node `o-a`, mark  $(0,1)$  visited. - From  $(0,1)$ , neighbor  $(1,1) = 't'$ : node `o-a` has child `'t'` -> move to node `o-a-t`, mark  $(1,1)$  visited. - From  $(1,1)$ , neighbor  $(2,1) = 'h'$ : node `o-a-t` has child `'h'` -> move to node `o-a-t-h`, whose `word = "oath"` -> add "oath" to results, clear `word` to avoid a duplicate add, then backtrack all the way out restoring every visited cell.

## Brute force

Run an independent word-search DFS for every word in the list, checking each cell against that one word's letters.

```

public List<String> findWordsBrute(char[][] board, String[] words) {
    List<String> found = new ArrayList<>();
    for (String word : words) {
        for (int r = 0; r < board.length; r++) {
            for (int c = 0; c < board[0].length; c++) {
                if (dfsWord(board, r, c, word, 0)) {
                    found.add(word);
                    break; // no need to keep scanning for this word
                }
            }
        }
    }
    return found;
}

private boolean dfsWord(char[][] board, int r, int c, String word, int idx) {
    if (idx == word.length()) return true;
    if (r < 0 || r >= board.length || c < 0 || c >= board[0].length) return false;
    if (board[r][c] != word.charAt(idx)) return false;

    char tmp = board[r][c];
    board[r][c] = '#'; // mark visited
    boolean found = dfsWord(board, r + 1, c, word, idx + 1)
        || dfsWord(board, r - 1, c, word, idx + 1)
        || dfsWord(board, r, c + 1, word, idx + 1)
        || dfsWord(board, r, c - 1, word, idx + 1);
    board[r][c] = tmp; // restore
    return found;
}

```

This re-scans the entire board once per word — with  $N$  words and an  $m \times n$  board that's  $O(N \cdot m \cdot n \cdot 4^L)$  in the worst case, and it repeats work for words that share prefixes (e.g. "oat" and "oath" both re-walk `o -> a -> t` from scratch).

### Optimized (trie + single board sweep)

Insert all words into a trie once. DFS from every cell, but only follow a neighbor if the trie has a matching child — this shares work across words with common prefixes and lets you bail out the moment a prefix can't lead to any word.

```

public List<String> findWords(char[][] board, String[] words) {
    TrieNode root = new TrieNode();
    for (String word : words) {
        TrieNode node = root;
        for (char c : word.toCharArray()) {
            int i = c - 'a';
            if (node.children[i] == null) node.children[i] = new TrieNode();
            node = node.children[i];
        }
        node.word = word; // stash the full word at its end node
    }

    List<String> found = new ArrayList<>();
    for (int r = 0; r < board.length; r++) {
        for (int c = 0; c < board[0].length; c++) {
            dfs(board, r, c, root, found);
        }
    }
    return found;
}

private void dfs(char[][] board, int r, int c, TrieNode node, List<String> found) {
    if (r < 0 || r >= board.length || c < 0 || c >= board[0].length) return;
    char ch = board[r][c];
    if (ch == '#' || node.children[ch - 'a'] == null) return; // no matching trie branch

    TrieNode next = node.children[ch - 'a'];
    if (next.word != null) {
        found.add(next.word);
        next.word = null; // avoid duplicate additions
    }

    board[r][c] = '#'; // mark visited
    dfs(board, r + 1, c, next, found);
    dfs(board, r - 1, c, next, found);
    dfs(board, r, c + 1, next, found);
    dfs(board, r, c - 1, next, found);
    board[r][c] = ch; // restore
}

```

TrieNode here needs one extra field: `String word`; (set at the end node instead of a plain `isEnd` boolean, so a match can be reported directly).

The trie beats searching each word separately for two reasons: (1) shared prefixes are walked **once** for all words that start with them, instead of once per word, and (2) the DFS can stop the moment a path leaves the trie (no word starts this way), pruning dead branches early instead of exploring them for every single word in the list.

| Version                     | Time                                                             | Space                       |
|-----------------------------|------------------------------------------------------------------|-----------------------------|
| Search each word separately | $O(N \cdot m \cdot n \cdot 4^L)$                                 | $O(L)$ recursion stack      |
| Trie + single sweep         | $O(m \cdot n \cdot 4^L)$ board sweep + $O(N \cdot L)$ trie build | $O(N \cdot L)$ for the trie |



## 3. Dynamic Programming (Advanced)

---

**The core idea.** Dynamic programming solves a problem by breaking it into smaller **subproblems**, solving each subproblem once, and reusing those answers instead of recomputing them. You define what a subproblem *means* (e.g. " $dp[i][j]$  = the answer for the first  $i$  characters of  $A$  and the first  $j$  characters of  $B$ "), find a **recurrence** that expresses a subproblem in terms of smaller ones, and pin down a **base case** that stops the recursion. A **1D table** is enough when a subproblem only depends on "how far along a single sequence am I" (e.g. index  $i$ ); a **2D table** is needed when the state depends on two independent positions (e.g. index into string  $A$  and index into string  $B$ , or a row and a column in a grid). You can fill the table two ways: **top-down memo** — write the natural recursion, cache each (state)  $\rightarrow$  answer the first time it's computed, and reuse the cache on repeat calls; or **bottom-up tabulation** — fill the table iteratively starting from the base case, in an order that guarantees every value a cell needs is already computed. Bottom-up is usually a few lines more code but avoids recursion overhead and often lets you compress the table to use less space.

**Reach for it when:** you're asked to **count** the number of ways to do something, find a **minimum/maximum** cost or length, or decide **feasibility** ("can this be done at all?"), and the brute-force recursion **recomputes the same subproblem** many times (overlapping subproblems) while also exhibiting **optimal substructure** (the best whole-solution is built from best solutions to its pieces).

---

### 3.1 Unique Paths

**Problem.** A robot sits at the top-left corner of an  $m \times n$  grid and can only move **down** or **right** at each step. Given the grid dimensions  $m$  and  $n$ , return the number of distinct paths from the top-left corner to the bottom-right corner.

- **Constraints (typical):**  $1 \leq m, n \leq 100$ . The answer fits in a 32-bit signed integer for these bounds.
- **Clarifying assumptions:** the grid has no obstacles or blocked cells (a common follow-up variant adds them — see "Obstacle" variants); moves are restricted to exactly down or right, never up or left; a  $1 \times 1$  grid has exactly one path (staying put counts as reaching the destination).
- **Why it's tricky:** it looks like a counting/combinatorics problem (and indeed  $C(m+n-2, m-1)$  solves it directly), but the DP framing is what interviewers want — it generalizes immediately to the obstacle variant, where the closed-form combinatorial solution breaks down.

**Example.**

```
Input:  m = 3, n = 7
Output: 28
```

**Intuition (diagram).** The number of ways to reach cell  $(i, j)$  is the sum of the ways to reach the cell above it and the cell to its left — those are the only two moves that could have landed you there. The first row and first column can only be reached one way (a straight line), so they seed the table with 1s.

```
dp[i][j] = dp[i-1][j] + dp[i][j-1]
```

```
      j=0 j=1 j=2 j=3
i=0 [ 1  1  1  1 ]
i=1 [ 1  2  3  4 ]
i=2 [ 1  3  6 10 ]   <- answer for a 3x4 grid
```

## Approach (step by step)

- **State:**  $dp[i][j]$  = the number of distinct paths from  $(0, 0)$  to cell  $(i, j)$ .
- **Recurrence in words:** the only ways to arrive at  $(i, j)$  are from directly above (having just moved down) or directly to the left (having just moved right), so the count at  $(i, j)$  is simply the sum of those two neighbors' counts.
- **Base case:** every cell in row  $0$  and every cell in column  $0$  has exactly one path — a straight line of moves in a single direction — so they're all seeded with  $1$ .
- **Fill order:** row by row, left to right (or column by column — either works), since  $dp[i][j]$  only needs the cell above  $(i-1)$  and the cell to the left  $(j-1)$ , both of which are already filled by the time you reach  $(i, j)$ .
- **Worked trace (3x4 grid, matching the diagram above):** row  $i=0$  and column  $j=0$  start at all 1s. Then  $dp[1][1] = dp[0][1] + dp[1][0] = 1 + 1 = 2$ ,  $dp[1][2] = dp[0][2] + dp[1][1] = 1 + 2 = 3$ , and so on, until  $dp[2][3] = dp[1][3] + dp[2][2] = 4 + 6 = 10$  — the final answer for this smaller 3x4 example.

## Brute force

Recurse from  $(0,0)$ , trying "down" and "right" at every cell.

```
public int uniquePathsBrute(int m, int n) {
    return countPaths(0, 0, m, n);
}

private int countPaths(int row, int col, int m, int n) {
    if (row == m - 1 && col == n - 1) return 1;    // reached the target
    if (row >= m || col >= n) return 0;           // fell off the grid
    return countPaths(row + 1, col, m, n) + countPaths(row, col + 1, m, n);
}
```

## Optimized (bottom-up tabulation)

**Recurrence in plain words:** the number of ways to reach a cell equals the ways to reach the cell above it plus the ways to reach the cell to its left; the top row and left column are all 1 because there is only one straight-line way to reach them.

```

public int uniquePaths(int m, int n) {
    int[][] dp = new int[m][n];
    for (int i = 0; i < m; i++) dp[i][0] = 1;    // left column: only one way (straight down)
    for (int j = 0; j < n; j++) dp[0][j] = 1;    // top row: only one way (straight right)

    for (int i = 1; i < m; i++) {
        for (int j = 1; j < n; j++) {
            dp[i][j] = dp[i - 1][j] + dp[i][j - 1];
        }
    }
    return dp[m - 1][n - 1];
}

```

| Version     | Time           | Space                    |
|-------------|----------------|--------------------------|
| Brute force | $O(2^{(m+n)})$ | $O(m+n)$ recursion stack |
| Tabulation  | $O(m \cdot n)$ | $O(m \cdot n)$           |

## 3.2 Minimum Path Sum

**Problem.** Given an  $m \times n$  grid `grid` where each cell holds a non-negative integer cost, find a path from the top-left corner to the bottom-right corner — moving only **down** or **right** at each step — that minimizes the sum of the numbers along the path. Return that minimum sum.

- **Constraints (typical):**  $1 \leq m, n \leq 200$ ,  $0 \leq \text{grid}[i][j] \leq 100$ . Values are non-negative, so there's no benefit to "backtracking" even if it were allowed.
- **Clarifying assumptions:** the starting and ending cells' own values are included in the sum; the grid is fully accessible (no obstacles/walls); a  $1 \times 1$  grid's answer is just that cell's value.
- **Why it's tricky:** it's structurally identical to Unique Paths (same movement rule, same 2D recurrence shape) but swaps "count the ways" for "minimize the cost" — recognizing that family resemblance is the key insight; the trap is forgetting that the first row/column need a *running sum* seed rather than a constant.

**Example.**

```

Input:  [[1,3,1],
         [1,5,1],
         [4,2,1]]
Output: 7          (path 1 -> 3 -> 1 -> 1 -> 1)

```

**Intuition (diagram).** To reach a cell as cheaply as possible, arrive from whichever neighbor (above or left) was cheaper, then add this cell's own cost. The first row and first column have only one way in, so their running totals are just prefix sums.

```
dp[i][j] = grid[i][j] + min(dp[i-1][j], dp[i][j-1])
```

|                                                                                                                                                                                                                                                                                |     |     |     |     |     |     |   |     |     |     |   |     |     |     |   |     |    |                                                                                                                                                                                                                                                                                |  |     |     |     |     |     |   |     |     |     |   |     |     |     |   |     |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|-----|-----|-----|-----|-----|---|-----|-----|-----|---|-----|-----|-----|---|-----|----|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|-----|-----|-----|-----|-----|---|-----|-----|-----|---|-----|-----|-----|---|-----|
| grid:                                                                                                                                                                                                                                                                          |     | dp: |     |     |     |     |   |     |     |     |   |     |     |     |   |     |    |                                                                                                                                                                                                                                                                                |  |     |     |     |     |     |   |     |     |     |   |     |     |     |   |     |
| <table border="0"> <tr> <td></td> <td>j=0</td> <td>j=1</td> <td>j=2</td> </tr> <tr> <td>i=0</td> <td>[ 1</td> <td>3</td> <td>1 ]</td> </tr> <tr> <td>i=1</td> <td>[ 1</td> <td>5</td> <td>1 ]</td> </tr> <tr> <td>i=2</td> <td>[ 4</td> <td>2</td> <td>1 ]</td> </tr> </table> |     | j=0 | j=1 | j=2 | i=0 | [ 1 | 3 | 1 ] | i=1 | [ 1 | 5 | 1 ] | i=2 | [ 4 | 2 | 1 ] | -> | <table border="0"> <tr> <td></td> <td>j=0</td> <td>j=1</td> <td>j=2</td> </tr> <tr> <td>i=0</td> <td>[ 1</td> <td>4</td> <td>5 ]</td> </tr> <tr> <td>i=1</td> <td>[ 2</td> <td>7</td> <td>6 ]</td> </tr> <tr> <td>i=2</td> <td>[ 6</td> <td>8</td> <td>7 ]</td> </tr> </table> |  | j=0 | j=1 | j=2 | i=0 | [ 1 | 4 | 5 ] | i=1 | [ 2 | 7 | 6 ] | i=2 | [ 6 | 8 | 7 ] |
|                                                                                                                                                                                                                                                                                | j=0 | j=1 | j=2 |     |     |     |   |     |     |     |   |     |     |     |   |     |    |                                                                                                                                                                                                                                                                                |  |     |     |     |     |     |   |     |     |     |   |     |     |     |   |     |
| i=0                                                                                                                                                                                                                                                                            | [ 1 | 3   | 1 ] |     |     |     |   |     |     |     |   |     |     |     |   |     |    |                                                                                                                                                                                                                                                                                |  |     |     |     |     |     |   |     |     |     |   |     |     |     |   |     |
| i=1                                                                                                                                                                                                                                                                            | [ 1 | 5   | 1 ] |     |     |     |   |     |     |     |   |     |     |     |   |     |    |                                                                                                                                                                                                                                                                                |  |     |     |     |     |     |   |     |     |     |   |     |     |     |   |     |
| i=2                                                                                                                                                                                                                                                                            | [ 4 | 2   | 1 ] |     |     |     |   |     |     |     |   |     |     |     |   |     |    |                                                                                                                                                                                                                                                                                |  |     |     |     |     |     |   |     |     |     |   |     |     |     |   |     |
|                                                                                                                                                                                                                                                                                | j=0 | j=1 | j=2 |     |     |     |   |     |     |     |   |     |     |     |   |     |    |                                                                                                                                                                                                                                                                                |  |     |     |     |     |     |   |     |     |     |   |     |     |     |   |     |
| i=0                                                                                                                                                                                                                                                                            | [ 1 | 4   | 5 ] |     |     |     |   |     |     |     |   |     |     |     |   |     |    |                                                                                                                                                                                                                                                                                |  |     |     |     |     |     |   |     |     |     |   |     |     |     |   |     |
| i=1                                                                                                                                                                                                                                                                            | [ 2 | 7   | 6 ] |     |     |     |   |     |     |     |   |     |     |     |   |     |    |                                                                                                                                                                                                                                                                                |  |     |     |     |     |     |   |     |     |     |   |     |     |     |   |     |
| i=2                                                                                                                                                                                                                                                                            | [ 6 | 8   | 7 ] |     |     |     |   |     |     |     |   |     |     |     |   |     |    |                                                                                                                                                                                                                                                                                |  |     |     |     |     |     |   |     |     |     |   |     |     |     |   |     |

<- answer = dp[2][2] = 7

## Approach (step by step)

- **State:**  $dp[i][j]$  = the minimum cost of any path from  $(0, 0)$  to  $(i, j)$ .
- **Recurrence in words:** the cheapest way to reach  $(i, j)$  is to arrive from whichever neighbor — the cell above or the cell to the left — has the smaller accumulated cost, then pay this cell's own cost on top of that.
- **Base case:**  $dp[0][0] = grid[0][0]$  (no cost precedes the start). Every other cell in row 0 can only be reached by moving right the whole way, so  $dp[0][j] = dp[0][j-1] + grid[0][j]$ ; symmetrically for column 0.
- **Fill order:** row by row, left to right, so both  $dp[i-1][j]$  (above) and  $dp[i][j-1]$  (left) are ready before  $dp[i][j]$  is computed.
- **Worked trace (on the example above):** row 0 becomes a running sum: 1, 4, 5. Column 0 becomes 1, 2, 6. Then  $dp[1][1] = grid[1][1] + \min(dp[0][1], dp[1][0]) = 5 + \min(4, 2) = 7$ ;  $dp[1][2] = 1 + \min(dp[0][2], dp[1][1]) = 1 + \min(5, 7) = 6$ ;  $dp[2][1] = 2 + \min(dp[1][1], dp[2][0]) = 2 + \min(7, 6) = 8$ ; finally  $dp[2][2] = 1 + \min(dp[1][2], dp[2][1]) = 1 + \min(6, 8) = 7$  — the answer.

## Brute force

Try both moves from every cell, return the cheapest path found.

```
public int minPathSumBrute(int[][] grid) {
    return minPath(grid, 0, 0);
}

private int minPath(int[][] grid, int row, int col) {
    int m = grid.length, n = grid[0].length;
    if (row == m - 1 && col == n - 1) return grid[row][col];
    if (row == m - 1) return grid[row][col] + minPath(grid, row, col + 1);
    if (col == n - 1) return grid[row][col] + minPath(grid, row + 1, col);
    return grid[row][col] + Math.min(minPath(grid, row + 1, col), minPath(grid, row, col + 1));
}
```

## Optimized (bottom-up tabulation)

**Recurrence in plain words:** the cheapest way to reach a cell is that cell's own cost plus whichever of "cheapest way to the cell above" or "cheapest way to the cell to the left" is smaller; the top-left cell costs just itself, and the rest of the top row / left column are running prefix sums.

```

public int minPathSum(int[][] grid) {
    int m = grid.length, n = grid[0].length;
    int[][] dp = new int[m][n];
    dp[0][0] = grid[0][0];

    for (int j = 1; j < n; j++) dp[0][j] = dp[0][j - 1] + grid[0][j]; // prefix sum, top
    for (int i = 1; i < m; i++) dp[i][0] = dp[i - 1][0] + grid[i][0]; // prefix sum, left

    for (int i = 1; i < m; i++) {
        for (int j = 1; j < n; j++) {
            dp[i][j] = grid[i][j] + Math.min(dp[i - 1][j], dp[i][j - 1]);
        }
    }
    return dp[m - 1][n - 1];
}

```

| Version     | Time           | Space                    |
|-------------|----------------|--------------------------|
| Brute force | $O(2^{(m+n)})$ | $O(m+n)$ recursion stack |
| Tabulation  | $O(m \cdot n)$ | $O(m \cdot n)$           |

### 3.3 Longest Common Subsequence

**Problem.** Given two strings `text1` and `text2`, return the length of their longest common **subsequence** — a sequence of characters that appears in both strings in the same relative order, but not necessarily contiguously (unlike a substring). If there is no common subsequence, return `0`.

- **Constraints (typical):** `1 <= text1.length, text2.length <= 1000`; both strings consist of lowercase English letters (or ASCII printable characters in some variants).
- **Clarifying assumptions:** you only need the *length* of the LCS, not the subsequence itself (a common follow-up asks you to reconstruct it by walking back through the table); the empty string is a subsequence of everything, so the answer is never negative.
- **Why it's tricky:** candidates often confuse "subsequence" with "substring" — a subsequence skips characters but never reorders them, so it needs a 2D state (one index per string) rather than the 1D state a contiguous-substring problem would use.

**Example.**

```

Input: text1 = "abcde", text2 = "ace"
Output: 3           ("ace" is a subsequence of both)

```

**Intuition (diagram).** This is the canonical two-string table: row `i` tracks a prefix of `text1`, column `j` tracks a prefix of `text2`. If the two current characters match, extend the best answer found without them by one. If they don't match, take the better of dropping one character from either string.

```

dp[i][j] = dp[i-1][j-1] + 1      if A[i-1] == B[j-1]
        = max(dp[i-1][j], dp[i][j-1]) otherwise

    ""  a  c  e
    "" [ 0  0  0  0 ]
a  [ 0  1  1  1 ]
b  [ 0  1  1  1 ]
c  [ 0  1  2  2 ]
d  [ 0  1  2  2 ]
e  [ 0  1  2  3 ]  <- answer = dp[5][3] = 3

```

## Approach (step by step)

- **State:**  $dp[i][j]$  = the length of the LCS of the first  $i$  characters of `text1` and the first  $j$  characters of `text2`. Row/column index  $0$  represents the *empty* prefix.
- **Recurrence in words:** compare the last character of each prefix — `text1[i-1]` and `text2[j-1]`. If they match, that shared character extends the best subsequence found without it, so add one to  $dp[i-1][j-1]$  (shrink both prefixes by one). If they don't match, that character can't be the end of the LCS, so try dropping it from one side or the other and keep the better result:  $\max(dp[i-1][j], dp[i][j-1])$ .
- **Base case:**  $dp[i][0] = 0$  and  $dp[0][j] = 0$  for all  $i, j$  — an empty prefix has no common subsequence with anything.
- **Fill order:** row by row (or column by column), increasing  $i$  and  $j$ , since  $dp[i][j]$  only reads  $dp[i-1][j-1]$ ,  $dp[i-1][j]$ , and  $dp[i][j-1]$  — all three already computed.
- **Worked trace ( `text1 = "abcde"`, `text2 = "ace"`, following the grid above row by row):** row `a` matches column `a`, so  $dp[1][1] = dp[0][0] + 1 = 1$ ; the rest of row `a` just carries that `1` forward since no later column-char matches `a` again. Row `c` ( $i=3$ ) matches column `c` ( $j=2$ ):  $dp[3][2] = dp[2][1] + 1 = 1 + 1 = 2$ . Row `e` ( $i=5$ ) matches column `e` ( $j=3$ ):  $dp[5][3] = dp[4][2] + 1 = 2 + 1 = 3$  — the final answer, corresponding to the subsequence "ace".

## Brute force

Recurse on both string positions, branching on match / no-match.

```

public int lcsBrute(String a, String b) {
    return lcs(a, b, 0, 0);
}

private int lcs(String a, String b, int i, int j) {
    if (i == a.length() || j == b.length()) return 0;
    if (a.charAt(i) == b.charAt(j)) {
        return 1 + lcs(a, b, i + 1, j + 1);
    }
    return Math.max(lcs(a, b, i + 1, j), lcs(a, b, i, j + 1));
}

```

## Optimized (bottom-up tabulation)

**Recurrence in plain words:** if the characters at the current prefixes match, the answer is one plus the best answer for both prefixes shortened by one; if they don't match, the answer is the better of dropping

the last character of `text1` or dropping the last character of `text2`. An empty prefix always contributes 0.

```
public int longestCommonSubsequence(String text1, String text2) {
    int m = text1.length(), n = text2.length();
    int[][] dp = new int[m + 1][n + 1]; // dp[i][j]: LCS of first i chars of text1, first j chars of text2

    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            if (text1.charAt(i - 1) == text2.charAt(j - 1)) {
                dp[i][j] = dp[i - 1][j - 1] + 1;
            } else {
                dp[i][j] = Math.max(dp[i - 1][j], dp[i][j - 1]);
            }
        }
    }
    return dp[m][n];
}
```

| Version     | Time           | Space                    |
|-------------|----------------|--------------------------|
| Brute force | $O(2^{(m+n)})$ | $O(m+n)$ recursion stack |
| Tabulation  | $O(m \cdot n)$ | $O(m \cdot n)$           |

### 3.4 Edit Distance

**Problem.** Given two strings `word1` and `word2`, return the minimum number of single- character **insert**, **delete**, or **replace** operations needed to transform `word1` into `word2`.

- **Constraints (typical):**  $0 \leq \text{word1.length}, \text{word2.length} \leq 500$ ; strings consist of lowercase English letters. Either string may be empty.
- **Clarifying assumptions:** each operation (insert/delete/replace) costs exactly 1, regardless of which characters are involved; operations may be applied in any order and any number of times; you only need the minimum *count*, not the actual sequence of edits (a common follow-up asks for the edit script itself, reconstructed by walking the table backward).
- **Why it's tricky:** three possible operations per mismatched pair (instead of LCS's two) makes the recurrence easy to get wrong — the trap is mismatching which neighbor corresponds to which operation (insert vs. delete are easy to swap), and forgetting that matching characters still require inheriting the diagonal, not skipping any cost check.

#### Example.

```
Input: word1 = "horse", word2 = "ros"
Output: 3          (horse -> rorse -> rose -> ros)
```

**Intuition (diagram).**  $dp[i][j]$  is the cost to convert the first  $i$  characters of `word1` into the first  $j$  characters of `word2`. If the last characters already match, no new operation is needed — inherit the diagonal. Otherwise pick the cheapest of three moves: **insert** (borrow from  $dp[i][j-1]$ , add the char

word2 needs), **delete** (borrow from `dp[i-1][j]`, drop a char from `word1`), or **replace** (borrow from `dp[i-1][j-1]`, swap one char), each costing +1.

```

      insert <- dp[i][j-1]
      \
delete -- dp[i-1][j]    dp[i][j]
      |      /
      dp[i-1][j-1] -- replace (diagonal)

    ""  r  o  s
"" [ 0  1  2  3 ]
h  [ 1  1  2  3 ]
o  [ 2  2  1  2 ]
r  [ 3  2  2  2 ]
s  [ 4  3  3  2 ]
e  [ 5  4  4  3 ]  <- answer = dp[5][3] = 3

```

## Approach (step by step)

- **State:** `dp[i][j]` = the minimum number of operations to convert the first `i` characters of `word1` into the first `j` characters of `word2`. Index `0` on either axis means the empty prefix.
- **Recurrence in words:** compare `word1[i-1]` to `word2[j-1]`. If they're the same character, no operation is spent on this pair — the cost is exactly the cost of converting the two strings *without* their last character, `dp[i-1][j-1]`. If they differ, one operation must be spent, and you take the cheapest of three ways to spend it: **insert** a character into `word1` to match `word2[j-1]` (cost = `dp[i][j-1] + 1`, since `word1`'s prefix is unchanged but `word2`'s target prefix shrinks by one), **delete** the last character of `word1` (cost = `dp[i-1][j] + 1`, `word1`'s prefix shrinks, `word2`'s doesn't), or **replace** the last character of `word1` with the one `word2` needs (cost = `dp[i-1][j-1] + 1`, both prefixes shrink).
- **Base case:** `dp[i][0] = i` (delete all `i` characters of `word1` to reach the empty string) and `dp[0][j] = j` (insert all `j` characters of `word2` into an empty string).
- **Fill order:** row by row, left to right, since `dp[i][j]` depends only on `dp[i-1][j-1]` (diagonal), `dp[i-1][j]` (above), and `dp[i][j-1]` (left) — all already filled.
- **Worked trace (word1 = "horse", word2 = "ros", tracking a few cells of the grid above):** row `h` vs column `r`: `h != r`, so `dp[1][1] = 1 + min(dp[0][1], dp[1][0], dp[0][0]) = 1 + min(1, 1, 0) = 1`. Row `o` vs column `r`: `dp[2][1] = 1 + min(dp[1][1], dp[2][0], dp[1][0]) = 1 + min(1, 2, 1) = 2`. Row `o` vs column `o` (`j=2`): `o == o` matches, so `dp[2][2] = dp[1][1] = 1` (inherit the diagonal, no new cost). Continuing this down to row `e` (`i=5`), column `s` (`j=3`) lands on `dp[5][3] = 3`, matching the example's 3-edit path: insert `r` ("horse" -> "rorse"), delete the extra `r` ("rorse" -> "rose"), delete `e` ("rose" -> "ros").

## Brute force

Recurse on both indices, trying all three edits when characters differ.



```

public int editDistanceBrute(String word1, String word2) {
    return edit(word1, word2, 0, 0);
}

private int edit(String w1, String w2, int i, int j) {
    if (i == w1.length()) return w2.length() - j;    // insert the rest of w2
    if (j == w2.length()) return w1.length() - i;    // delete the rest of w1
    if (w1.charAt(i) == w2.charAt(j)) {
        return edit(w1, w2, i + 1, j + 1);           // chars match, no cost
    }
    int insert = 1 + edit(w1, w2, i, j + 1);
    int delete = 1 + edit(w1, w2, i + 1, j);
    int replace = 1 + edit(w1, w2, i + 1, j + 1);
    return Math.min(insert, Math.min(delete, replace));
}

```

## Optimized (bottom-up tabulation)

**Recurrence in plain words:** if the current characters match, the cost is the same as converting both strings without their last character (no new operation). If they don't match, take 1 plus the cheapest of: insert (use `dp[i][j-1]`), delete (use `dp[i-1][j]`), or replace (use `dp[i-1][j-1]`). Converting an empty string costs one insert/delete per remaining character, which seeds the first row and column.

```

public int minDistance(String word1, String word2) {
    int m = word1.length(), n = word2.length();
    int[][] dp = new int[m + 1][n + 1];

    for (int i = 0; i <= m; i++) dp[i][0] = i;    // delete all of word1's first i chars
    for (int j = 0; j <= n; j++) dp[0][j] = j;    // insert all of word2's first j chars

    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            if (word1.charAt(i - 1) == word2.charAt(j - 1)) {
                dp[i][j] = dp[i - 1][j - 1];
            } else {
                int insert = dp[i][j - 1];
                int delete = dp[i - 1][j];
                int replace = dp[i - 1][j - 1];
                dp[i][j] = 1 + Math.min(insert, Math.min(delete, replace));
            }
        }
    }
    return dp[m][n];
}

```

| Version     | Time           | Space                    |
|-------------|----------------|--------------------------|
| Brute force | $O(3^{(m+n)})$ | $O(m+n)$ recursion stack |
| Tabulation  | $O(m \cdot n)$ | $O(m \cdot n)$           |

### 3.5 Partition Equal Subset Sum

**Problem.** Given an array `nums` of positive integers, determine if the array can be partitioned into two subsets such that the sum of the elements in both subsets is equal. Return `true` or `false`.

- **Constraints (typical):** `1 <= nums.length <= 200`, `1 <= nums[i] <= 100`. All values are positive (no zeros or negatives), which keeps the total sum well-defined and bounded.
- **Clarifying assumptions:** every element must be used in exactly one of the two subsets (no element is discarded); duplicate values are allowed and are treated as distinct items even if equal in value; a single-element array can never be split evenly unless that element is `0` (excluded here since values are positive), so it's always `false` for `nums.length == 1`.
- **Why it's tricky:** it looks like it needs to enumerate subsets ( $2^n$ ), but it reduces to a **0/1 knapsack** feasibility question — can some subset hit exactly `total / 2`? The second trap is the iteration direction: iterating the sum axis forward would let a single number be "reused" multiple times within the same subset, which is wrong for 0/1 knapsack.

**Example.**

```
Input: [1, 5, 11, 5]
Output: true          (subsets [1, 5, 5] and [11], both sum to 11)
```

**Intuition (diagram).** Equal-sum partition is possible exactly when some subset sums to `total / 2` (if `total` is odd, it's immediately impossible). This is 0/1 knapsack: for each number, decide include-or-skip, tracking which *sums* are reachable. `dp[s]` = true if some subset of the numbers considered so far sums to exactly `s`. Iterate the sum dimension **downward** so each number is only used once per subset.

```
target = 11, nums = [1, 5, 11, 5]

sum:      0  1  2  3  4  5  6  7  8  9 10 11
start: [ T  F  F  F  F  F  F  F  F  F  F  F ]
+1:      [ T  T  F  F  F  F  F  F  F  F  F  F ]
+5:      [ T  T  F  F  F  T  T  F  F  F  F  F ]
+11:     [ T  T  F  F  F  T  T  F  F  F  F  T ] <- dp[11] = true
+5:      [ T  T  F  F  F  T  T  F  F  F  T  T ] <- also reached via 1+5+5
```

**Approach (step by step)**

- **State:** `dp[s]` = true if some subset of the numbers processed so far sums to exactly `s`. The full array of `s` values, `0..target`, together represent "which totals are currently reachable."
- **Key idea / invariant:** process numbers one at a time; after processing a number, `dp[s]` is true if it was reachable *without* that number, or if `s` minus that number was reachable *before* adding it (i.e., adding this number to some smaller already-reachable sum lands exactly on `s`). Scanning `s` from `target` down to the number's value (rather than up) guarantees you read `dp[s - num]` from *before* the current number was folded in, so each number is used at most once per subset.
- **Base case:** `dp[0] = true` before processing anything — the empty subset always sums to
- First check `total % 2 != 0`, which makes an equal split impossible outright.
- **Fill order:** outer loop over each number in `nums`; inner loop over `s` from `target` down to that number's value, updating `dp[s] |= dp[s - num]`.

- **Worked trace ( `nums = [1, 5, 11, 5]` , `target = 11` , matching the diagram):** start with only `dp[0] = true` . After folding in `1` : `dp[1]` becomes true. After folding in `5` : `dp[5]` and `dp[6]` (= `1+5`) become true. After folding in `11` : `dp[11]` becomes true directly — the answer is already true at this point. After folding in the second `5` (for completeness): `dp[10]` (= `5+5`) also becomes true, showing an alternate reachable path.

## Brute force

Recurse: for each number, try including it in "subset A" or not.

```
public boolean canPartitionBrute(int[] nums) {
    int total = Arrays.stream(nums).sum();
    if (total % 2 != 0) return false;
    return canReach(nums, 0, total / 2);
}

private boolean canReach(int[] nums, int index, int remaining) {
    if (remaining == 0) return true;
    if (index == nums.length || remaining < 0) return false;
    // either use nums[index] toward the target, or skip it
    return canReach(nums, index + 1, remaining - nums[index]) || canReach(nums, index + 1,
}
```

## Optimized (1D boolean tabulation)

**Recurrence in plain words:** a sum `s` is reachable after considering a number `x` if it was already reachable without `x`, or if `s - x` was reachable before `x` was added (i.e. add `x` to a previously reachable smaller sum). Walking the sum axis from high to low guarantees each number is applied at most once.

```
public boolean canPartition(int[] nums) {
    int total = Arrays.stream(nums).sum();
    if (total % 2 != 0) return false;
    int target = total / 2;

    boolean[] dp = new boolean[target + 1];
    dp[0] = true; // sum 0 is always reachable (pick nothing)

    for (int num : nums) {
        for (int s = target; s >= num; s--) { // go downward: use each number once
            if (dp[s - num]) {
                dp[s] = true;
            }
        }
    }
    return dp[target];
}
```

| Version     | Time                       | Space                  |
|-------------|----------------------------|------------------------|
| Brute force | $O(2^n)$                   | $O(n)$ recursion stack |
| Tabulation  | $O(n \cdot \text{target})$ | $O(\text{target})$     |

## 3.6 Maximal Square

**Problem.** Given an  $m \times n$  binary matrix filled with '0' s and '1' s, find the largest square containing only 1 s and return its **area**.

- **Constraints (typical):**  $1 \leq m, n \leq 300$ ; `matrix[i][j]` is either '0' or '1' (often given as `char`, not `int`, in the LeetCode signature).
- **Clarifying assumptions:** "square" means axis-aligned (no rotation); a single 1 cell counts as a valid  $1 \times 1$  square (area 1); if the matrix is all 0 s, the answer is 0.
- **Why it's tricky:** the natural first instinct is to check, for every cell, how far a square can grow — an  $O((mn)^2)$  brute force. The DP insight is subtler than a simple prefix-sum: the side length achievable at  $(i, j)$  is capped by the *minimum* of the three neighboring side lengths (above, left, diagonal), not their sum or max — one weak neighbor bottlenecks the whole square.

**Example.**

```
Input:  [[1,0,1,0,0],
         [1,0,1,1,1],
         [1,1,1,1,1],
         [1,0,0,1,0]]
Output: 4          (a 2x2 square of 1s)
```

**Intuition (diagram).** `dp[i][j]` holds the side length of the largest all-1s square whose **bottom-right corner** is at  $(i, j)$ . A square can only extend to size  $k$  at  $(i, j)$  if the cell is a 1 **and** the square just above, just to the left, and diagonally above-left were all already at least  $k-1$  — the smallest of those three neighbors is the bottleneck.

```
dp[i][j] = min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1]) + 1    if matrix[i][j] == 1
           = 0                                              otherwise

matrix:
  j=0 j=1 j=2 j=3 j=4
i=0 [ 1  0  1  0  0 ]
i=1 [ 1  0  1  1  1 ]
i=2 [ 1  1  1  1  1 ]
i=3 [ 0  0  1  1  0 ]

dp:
  j=0 j=1 j=2 j=3 j=4
i=0 [ 1  0  1  0  0 ]
i=1 [ 1  0  1  1  1 ]
i=2 [ 1  1  1  2  2 ]
i=3 [ 0  0  1  2  0 ]  <- best side = 2, area = 4
```

### Approach (step by step)

- **State:** `dp[i][j]` = the side length of the largest all-1 s square whose bottom-right corner sits at  $(i, j)$ . Note this is a side length, not an area — the area is computed once at the end from the best side length seen.
- **Recurrence in words:** if `matrix[i][j]` is 0, no square can end here, so `dp[i][j] = 0`. If it's 1, the square ending at  $(i, j)$  can only be as large as the smallest of the three squares ending at its three neighbors — above, left, and diagonally above-left — plus one for this new row/column of cells. Any one weak neighbor caps the whole square (e.g. if the square above only reaches side 1, you can't suddenly have a side-3 square here even if left and diagonal both reach side 3).
- **Base case:** cells in row 0 or column 0 have no "above" or "left" neighbor to check, so `dp[i][j] = 1` whenever `matrix[i][j] == 1` there (a 1-cell can always form a  $1 \times 1$  square), and 0 otherwise.

- **Fill order:** row by row, left to right, since `dp[i][j]` only reads `dp[i-1][j]`, `dp[i][j-1]`, and `dp[i-1][j-1]` — all already computed by the time you reach `(i, j)`. Track a running `best` side length as you fill.
- **Worked trace (on the example matrix/dp grid above):** at `(i=2, j=2)`, `matrix[2][2] == 1` and its three neighbors are `dp[1][2] = 1`, `dp[2][1] = 1`, `dp[1][1] = 0` (the diagonal is a 0-cell), so `dp[2][2] = min(1, 1, 0) + 1 = 1` — the diagonal's 0 bottlenecks it back down to a 1x1 square even though both straight neighbors reach 1. Moving one cell right, at `(i=2, j=3)`: neighbors are `dp[1][3] = 1`, `dp[2][2] = 1`, `dp[1][2] = 1`, so `dp[2][3] = min(1, 1, 1) + 1 = 2` — the first 2x2 square appears here. Finally at `(i=3, j=3)`: neighbors are `dp[2][3] = 2`, `dp[3][2] = 1`, `dp[2][2] = 1`, so `dp[3][3] = min(2, 1, 1) + 1 = 2`, matching `best = 2` and area `2*2 = 4`.

## Brute force

For every cell, grow a square from it as far as possible.

```
public int maximalSquareBrute(char[][] matrix) {
    int m = matrix.length, n = matrix[0].length, best = 0;
    for (int i = 0; i < m; i++) {
        for (int j = 0; j < n; j++) {
            int side = 0;
            while (i + side < m && j + side < n && isFullSquare(matrix, i, j, side + 1)) {
                side++;
            }
            best = Math.max(best, side);
        }
    }
    return best * best;
}

private boolean isFullSquare(char[][] matrix, int row, int col, int side) {
    for (int i = row; i < row + side; i++) {
        for (int j = col; j < col + side; j++) {
            if (matrix[i][j] != '1') return false;
        }
    }
    return true;
}
```

## Optimized (bottom-up tabulation)

**Recurrence in plain words:** if the current cell is a 1, the biggest square ending here is one more than the smallest of the squares ending directly above, directly to the left, and diagonally above-left (all three must support the larger size). If the cell is a 0, no square can end there.

```

public int maximalSquare(char[][] matrix) {
    int m = matrix.length, n = matrix[0].length;
    int[][] dp = new int[m][n];
    int best = 0;

    for (int i = 0; i < m; i++) {
        for (int j = 0; j < n; j++) {
            if (matrix[i][j] == '1') {
                if (i == 0 || j == 0) {
                    dp[i][j] = 1; // edge of the grid: square of side 1
                } else {
                    dp[i][j] = 1 + Math.min(dp[i - 1][j - 1],
                                            Math.min(dp[i - 1][j], dp[i][j - 1]));
                }
                best = Math.max(best, dp[i][j]);
            }
        }
    }
    return best * best;
}

```

| Version     | Time               | Space          |
|-------------|--------------------|----------------|
| Brute force | $O((m \cdot n)^2)$ | $O(1)$         |
| Tabulation  | $O(m \cdot n)$     | $O(m \cdot n)$ |

### 3.7 Decode Ways

**Problem.** A message consisting of digits was encoded using the mapping 'A' -> "1", 'B' -> "2", ..., 'Z' -> "26". Given the encoded string `s` (containing only digits), return the number of ways it can be decoded. A '0' can never start a valid one-digit or two-digit group (there's no letter mapped to "0").

- **Constraints (typical):**  $1 \leq s.length \leq 100$ ; `s` consists only of digits, and may contain leading zeros in the input itself (which then make it undecodable in some or all ways).
- **Clarifying assumptions:** groupings are only 1 or 2 digits (never 3+, since 'Z' is the largest letter at 26); a two-digit group must be between "10" and "26" inclusive to be valid (e.g. "27" is not a valid two-digit group, and "05" is invalid because it has a leading zero); if `s` starts with '0', the answer is 0 (no valid decoding exists at all).
- **Why it's tricky:** it looks like Climbing Stairs (a 1D "take 1 or take 2 steps" recurrence) but each step is conditionally valid depending on the actual digits, not unconditionally — the leading-zero rule is the classic trap that breaks naive solutions on inputs like "06" or "100".

#### Example.

Input: "226"  
 Output: 3 ("2 2 6" -> BBF, "22 6" -> VF, "2 26" -> BZ)

**Intuition (diagram).** `dp[i]` = number of ways to decode the first `i` characters. From position `i`, you can peel off one digit (valid if it isn't '0') or two digits (valid if they form 10 – 26). Each valid peel adds

the ways to decode everything before it.

```
dp[i] = dp[i-1] (if s[i-1] != '0')
        + dp[i-2] (if s[i-2..i-1] is "10".."26")

s = "2 2 6"
dp[0] = 1 (empty prefix)
dp[1]: last 1 digit "2" valid single -> dp[1] = dp[0] = 1
dp[2]: last 1 digit "2" valid, last 2 "22" valid -> dp[2] = dp[1] + dp[0] = 1 + 1 = 2
dp[3]: last 1 digit "6" valid, last 2 "26" valid -> dp[3] = dp[2] + dp[1] = 2 + 1 = 3 <-
```

## Approach (step by step)

- **State:**  $dp[i]$  = the number of ways to decode the first  $i$  characters of  $s$  (a prefix).  $dp[0]$  represents the empty prefix.
- **Recurrence in words:** to decode the first  $i$  characters, the last "group" peeled off is either a single digit or a pair of digits. If the single digit at position  $i-1$  is not  $'0'$ , it's a valid group on its own, and it contributes all the ways to decode everything before it:  $dp[i-1]$ . Separately, if the two digits ending at position  $i-1$  (i.e.  $s[i-2..i-1]$ ) form a number between  $10$  and  $26$ , that's also a valid group, contributing  $dp[i-2]$ . Both contributions are added together since they're independent decodings.
- **Base case:**  $dp[0] = 1$  — there's exactly one way to decode nothing (do nothing).  $dp[1] = 1$  if  $s[0] \neq '0'$  (a single valid first character), otherwise the whole string is undecodable and the answer is  $0$ .
- **Fill order:** left to right, increasing  $i$  from  $2$  to  $n$ , since  $dp[i]$  only depends on  $dp[i-1]$  and  $dp[i-2]$ , both already computed.
- **Worked trace (  $s = "226"$  , matching the diagram above):**  $dp[0] = 1$ .  $dp[1]$ : digit  $'2'$  is non-zero, so  $dp[1] = dp[0] = 1$ .  $dp[2]$ : last single digit  $'2'$  is valid ( $+= dp[1] = 1$ ), and the last two digits  $"22"$  are between  $10$  and  $26$  ( $+= dp[0] = 1$ ), so  $dp[2] = 1 + 1 = 2$ .  $dp[3]$ : last single digit  $'6'$  is valid ( $+= dp[2] = 2$ ), and the last two digits  $"26"$  are between  $10$  and  $26$  ( $+= dp[1] = 1$ ), so  $dp[3] = 2 + 1 = 3$  — the final answer.

## Brute force

Recurse from each position, trying a one-digit and a two-digit cut.

```

public int numDecodingsBrute(String s) {
    return decode(s, 0);
}

private int decode(String s, int i) {
    if (i == s.length()) return 1;           // used up the whole string: one valid way
    if (s.charAt(i) == '0') return 0;        // leading zero: no valid group starts here

    int ways = decode(s, i + 1);             // take one digit
    if (i + 1 < s.length()) {
        int twoDigit = Integer.parseInt(s.substring(i, i + 2));
        if (twoDigit <= 26) {
            ways += decode(s, i + 2);        // take two digits
        }
    }
    return ways;
}

```

### Optimized (bottom-up tabulation)

**Recurrence in plain words:** the ways to decode the first `i` characters equals the ways to decode the first `i-1` characters (if the single digit at position `i-1` is nonzero) plus the ways to decode the first `i-2` characters (if the two digits ending at `i-1` form `10` through `26`). The empty prefix has exactly one way (do nothing).

```

public int numDecodings(String s) {
    int n = s.length();
    if (n == 0 || s.charAt(0) == '0') return 0;

    int[] dp = new int[n + 1];
    dp[0] = 1;           // empty prefix: one way
    dp[1] = 1;           // first char already checked non-'0' above

    for (int i = 2; i <= n; i++) {
        char oneDigitChar = s.charAt(i - 1);
        if (oneDigitChar != '0') {
            dp[i] += dp[i - 1];           // last single digit is a valid letter
        }
        int twoDigit = Integer.parseInt(s.substring(i - 2, i));
        if (twoDigit >= 10 && twoDigit <= 26) {
            dp[i] += dp[i - 2];           // last two digits form a valid letter
        }
    }
    return dp[n];
}

```

| Version     | Time     | Space                  |
|-------------|----------|------------------------|
| Brute force | $O(2^n)$ | $O(n)$ recursion stack |
| Tabulation  | $O(n)$   | $O(n)$                 |



### 3.8 Longest Palindromic Substring

**Problem.** Given a string `s`, return the longest substring of `s` that reads the same forwards and backwards (a palindrome). A substring is contiguous, unlike a subsequence.

- **Constraints (typical):** `1 <= s.length <= 1000`; `s` consists of printable ASCII characters (often restricted to letters and digits).
- **Clarifying assumptions:** if multiple longest palindromes of the same maximal length exist, returning any one of them is acceptable; a single character is trivially a palindrome of length 1, so the answer is never empty.
- **Why it's tricky:** the naive approach checks all  $O(n^2)$  substrings for the palindrome property in  $O(n)$  each, giving  $O(n^3)$  — the insight to drop a factor of `n` is that palindromes are built outward from a **center**, so instead of checking substrings you can *grow* them, verifying only the newly added boundary character at each step ( $O(1)$  per expansion) rather than re-checking the whole substring.

**Example.**

```
Input: "babad"
Output: "bab"      ("aba" is also a valid answer)
```

**Intuition (diagram).** A palindrome is symmetric around a center, and that center is either a single character (odd length) or the gap between two characters (even length). Expanding outward from every possible center and stopping the moment characters stop matching finds the longest palindrome in  $O(n^2)$  with  $O(1)$  extra space — no table needed.

```
s = "b a b a d"
  expand around center i=2 ('b'):
    L=2,R=2 "b"          match, grow
    L=1,R=3 "aba"        match, grow
    L=0,R=4 "babad" no   'b' != 'd' -> stop, best so far "aba" (len 3)

  expand around center i=1 ('a'):
    L=1,R=1 "a"
    L=0,R=2 "bab"        match, grow          <- also length 3
```

An equivalent (and common alternative) framing is a **2D interval DP** table where `dp[i][j] = true` if `s[i..j]` is a palindrome:

```
dp[i][j] = true          if i == j
              = s[i] == s[j]      if j == i + 1
              = s[i] == s[j] && dp[i+1][j-1] otherwise

  b a b a d
b [T F T F F ]
a [  T F T F ]
b [    T F F ]
a [      T F ]
d [        T ]  <- longest true span: "bab" (i=0..2) or "aba" (i=1..3), both len 3
```

## Approach (step by step)

This problem has two standard solutions; both are detailed below since either may come up.

**Expand-around-center approach (the optimized solution below):** - **Key idea / invariant:** every palindrome has a center — either a single character (for odd-length palindromes) or the gap between two adjacent characters (for even-length palindromes). There are exactly  $2n - 1$  such centers in a string of length  $n$ . For each center, grow a  $(lo, hi)$  pointer pair outward one step at a time; the invariant is that  $s[lo..hi]$  is a palindrome as long as the loop keeps running, and it stops the instant  $s[lo] \neq s[hi]$  or a boundary is hit. - **Numbered walk-through:** (1) for each index `center` from  $0$  to  $n-1$ : (2) expand from  $(center, center)$  to find the longest odd-length palindrome centered there; (3) expand from  $(center, center+1)$  to find the longest even-length palindrome centered on that gap; (4) if either expansion beats the best length seen so far, record its start index and length; (5) after checking all centers, slice out the best substring. - **Worked trace:** shown in the diagram above — centering on index 2 ( 'b' ) grows to "aba" (length 3) before 'b'  $\neq$  'd' stops it; centering on index 1 ( 'a' ) grows to "bab" (also length 3). Both are valid answers of the same maximal length.

**2D interval DP approach (the alternative table above):** - **State:**  $dp[i][j]$  = true if the substring  $s[i..j]$  (inclusive on both ends) is a palindrome. - **Recurrence in words:** a single character ( $i == j$ ) is always a palindrome. Two adjacent characters ( $j == i + 1$ ) form a palindrome exactly when they're equal. For any longer span,  $s[i..j]$  is a palindrome exactly when its two outer characters match **and** the inner substring  $s[i+1..j-1]$  is itself a palindrome — so the answer depends on a cell that is both one row down and one column left, i.e. closer to the diagonal. - **Base case:** every cell on the main diagonal ( $i == j$ ) is true. - **Fill order:** this is the subtle part —  $dp[i][j]$  depends on  $dp[i+1][j-1]$ , a cell with a *larger* row index than  $i$ . So you must fill by increasing **substring length** (or equivalently, process  $i$  from  $n-1$  down to  $0$ , and for each  $i$ ,  $j$  from  $i$  to  $n-1$ ), never simply row-by-row top-to-bottom. - **Worked trace (on the grid above,  $s = \text{"babad"}$ ):** all diagonal cells are T. For length-2 spans:  $dp[0][1]$  ("ba") is F since  $b \neq a$ . For length-3 spans:  $dp[0][2]$  ("bab") is true since  $s[0]==s[2]$  ( $b==b$ ) and the inner  $dp[1][1]$  is trivially true; similarly  $dp[1][3]$  ("aba") is true. No span of length 4 or 5 is a palindrome (checking  $dp[0][4]$  needs  $s[0]==s[4]$ , i.e.  $b==d$ , which fails), so the longest true span has length 3.

## Brute force

Check every substring for being a palindrome, keep the longest.

```

public String longestPalindromeBrute(String s) {
    String best = "";
    for (int i = 0; i < s.length(); i++) {
        for (int j = i; j < s.length(); j++) {
            String candidate = s.substring(i, j + 1);
            if (isPalindrome(candidate) && candidate.length() > best.length()) {
                best = candidate;
            }
        }
    }
    return best;
}

private boolean isPalindrome(String s) {
    int lo = 0, hi = s.length() - 1;
    while (lo < hi) {
        if (s.charAt(lo) != s.charAt(hi)) return false;
        lo++;
        hi--;
    }
    return true;
}

```

### Optimized (expand around center)

**Recurrence in plain words:** for every possible center (each index, and each gap between two indices), grow outward one step at a time while the outer characters still match; the widest successful growth from any center is the answer. No table is stored — only the best bounds seen so far.

```

public String longestPalindrome(String s) {
    if (s == null || s.isEmpty()) return "";
    int bestStart = 0, bestLen = 1;

    for (int center = 0; center < s.length(); center++) {
        int len1 = expand(s, center, center);          // odd-length palindromes
        int len2 = expand(s, center, center + 1);      // even-length palindromes
        int longer = Math.max(len1, len2);
        if (longer > bestLen) {
            bestLen = longer;
            bestStart = center - (longer - 1) / 2;
        }
    }
    return s.substring(bestStart, bestStart + bestLen);
}

// Expands outward from (lo, hi); returns the length of the palindrome found.
private int expand(String s, int lo, int hi) {
    while (lo >= 0 && hi < s.length() && s.charAt(lo) == s.charAt(hi)) {
        lo--;
        hi++;
    }
    return hi - lo - 1;
}

```

| Version     | Time     | Space                      |
|-------------|----------|----------------------------|
| Brute force | $O(n^3)$ | $O(n)$ per substring check |

| Version                      | Time     | Space    |
|------------------------------|----------|----------|
| 2D interval DP (alternative) | $O(n^2)$ | $O(n^2)$ |
| Expand around center         | $O(n^2)$ | $O(1)$   |

---

## 4. Mixed High-Frequency Set

---

**The core idea.** Some problems don't belong to a single named pattern — they're must-know classics that show up across interviews precisely because each one teaches a distinct trick: turning division into products, bounding a traversal early, using two pointers against a height profile, or sliding a fixed-size window of counts. Knowing these cold rounds out your pattern library.

**Reach for it when:** none of the other buckets fit cleanly — these are grab-bag classics that interviewers ask because they're short to state, sneaky to get optimal, and each one tests a different fundamental skill (prefix/suffix sums, tree traversal, two pointers, fixed windows).

For any tree problem below, assume this node definition:

```
class TreeNode {
    int val;
    TreeNode left, right;
    TreeNode(int v) { val = v; }
}
```

### 4.1 Product of Array Except Self

**Problem.** Given an integer array `nums` of length `n`, return an array `answer` of the same length where `answer[i]` equals the product of every element of `nums` except `nums[i]`. The catch: you must compute this in  $O(n)$  time **without using division**, and ideally in  $O(1)$  extra space (not counting the output array).

- **Constraints:**  $2 \leq n \leq 10^5$ ; `nums[i]` can be negative, zero, or positive ( $-30 \leq \text{nums}[i] \leq 30$  is a typical bound); the product of any prefix/suffix fits in a 32-bit int for the stated range. `nums` may contain zero or even multiple zeros.
- **Clarifying assumptions:** confirm whether division is truly disallowed (it is, per the classic statement) and whether more than one zero can appear (yes — if there are two or more zeros, every `answer[i]` is 0; if there's exactly one zero, only that index's answer is nonzero).
- **Why it's tricky:** the instinctive approach is "compute the total product, then divide by `nums[i]`" — but division is banned (and breaks on zeros anyway), so you need a way to get "everything except i" using only multiplication.

**Example.**

```
Input:  nums = [1, 2, 3, 4]
Output: [24, 12, 8, 6]
        (24 = 2*3*4, 12 = 1*3*4, 8 = 1*2*4, 6 = 1*2*3)
```

**Intuition (diagram).** `answer[i]` is just "everything to the left of i" times "everything to the right of i". Compute a running **prefix** product left-to-right, then fold in a running **suffix** product right-to-left. No division needed, and you can reuse the output array to avoid extra space.

|         |    |    |   |   |
|---------|----|----|---|---|
| index:  | 0  | 1  | 2 | 3 |
| nums:   | 1  | 2  | 3 | 4 |
| prefix: | 1  | 1  | 2 | 6 |
| suffix: | 24 | 12 | 4 | 1 |
| answer: | 24 | 12 | 8 | 6 |

```

prefix[i] = product of nums[0 .. i-1] (everything strictly left of i)
suffix[i] = product of nums[i+1 .. n-1] (everything strictly right of i)
answer[i] = prefix[i] * suffix[i]

```

## Approach (step by step)

**Key idea:**  $\text{answer}[i] = (\text{product of everything left of } i) * (\text{product of everything right of } i)$ . Both halves can be built incrementally in a single left-to-right and a single right-to-left pass, so the whole thing runs in  $O(n)$  with two multiplications per index and no division.

1. Allocate `answer` of length `n`. Set `answer[0] = 1` (there's nothing to the left of index 0).
2. **Left-to-right pass:** for `i` from 1 to `n-1`, set `answer[i] = answer[i-1] * nums[i-1]`. After this pass, `answer[i]` holds the prefix product — the product of everything strictly to the left of `i`.
3. **Right-to-left pass:** keep a running `suffix` variable, initialized to 1. For `i` from `n-1` down to 0: multiply `answer[i] *= suffix` (folding in the suffix product computed so far), then update `suffix *= nums[i]` (extend the running suffix to include `nums[i]` for the next, more-left index).
4. Return `answer`.

**Worked trace on** `nums = [1, 2, 3, 4]`:

*Left-to-right pass (building prefixes into `answer`):*

```

i=0: answer[0] = 1
i=1: answer[1] = answer[0] * nums[0] = 1 * 1 = 1
i=2: answer[2] = answer[1] * nums[1] = 1 * 2 = 2
i=3: answer[3] = answer[2] * nums[2] = 2 * 3 = 6
answer after pass 1: [1, 1, 2, 6] (these are the prefixes)

```

*Right-to-left pass (folding in suffixes, suffix starts at 1):*

```

i=3: answer[3] *= suffix(1) -> 6 * 1 = 6; suffix *= nums[3](4) -> suffix = 4
i=2: answer[2] *= suffix(4) -> 2 * 4 = 8; suffix *= nums[2](3) -> suffix = 12
i=1: answer[1] *= suffix(12) -> 1 * 12 = 12; suffix *= nums[1](2) -> suffix = 24
i=0: answer[0] *= suffix(24) -> 1 * 24 = 24; suffix *= nums[0](1) -> suffix = 24
answer after pass 2: [24, 12, 8, 6] matches expected output

```

## Brute force

For each index, multiply every other element — recompute from scratch each time.

```

public int[] productExceptSelfBrute(int[] nums) {
    int n = nums.length;
    int[] answer = new int[n];
    for (int i = 0; i < n; i++) {
        int product = 1;
        for (int j = 0; j < n; j++) {
            if (j != i) {
                product *= nums[j];
            }
        }
        answer[i] = product;
    }
    return answer;
}

```

### Optimized (prefix \* suffix products)

One left-to-right pass builds prefixes directly into the answer; one right-to-left pass multiplies in the suffixes.

```

public int[] productExceptSelf(int[] nums) {
    int n = nums.length;
    int[] answer = new int[n];

    // answer[i] starts as the product of everything to the left of i
    answer[0] = 1;
    for (int i = 1; i < n; i++) {
        answer[i] = answer[i - 1] * nums[i - 1];
    }

    // fold in the product of everything to the right of i
    int suffix = 1;
    for (int i = n - 1; i >= 0; i--) {
        answer[i] *= suffix;
        suffix *= nums[i];
    }

    return answer;
}

```

| Version         | Time     | Space                                     |
|-----------------|----------|-------------------------------------------|
| Brute force     | $O(n^2)$ | $O(1)$ extra                              |
| Prefix * suffix | $O(n)$   | $O(1)$ extra (output array doesn't count) |

## 4.2 Kth Smallest Element in a BST

**Problem.** Given the root of a binary search tree (BST) containing  $n$  nodes and an integer  $k$  (with  $1 \leq k \leq n$ ), return the  $k$ -th smallest value stored in the tree, where  $k = 1$  means the smallest value.

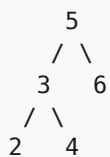
- **Constraints:**  $1 \leq n \leq 10^4$  nodes; node values fit in a 32-bit int and are typically assumed **distinct** (standard BST property in this problem's usual framing);  $k$  is always valid ( $1 \leq k \leq n$ )

n), so no out-of-range handling is required by the problem itself, though production code should still guard against it.

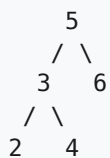
- **Clarifying assumptions:** confirm whether the tree can be modified concurrently between queries — if there are many repeated queries, it's worth asking whether augmenting each node with a subtree-size count (to answer in  $O(h)$  per query) is acceptable, versus the simpler single-query in-order approach shown here.
- **Why it's tricky:** the naive instinct is to collect *all* values and sort, throwing away the fact that a BST is already implicitly sorted by structure — the real insight is that in-order traversal visits nodes in sorted order "for free," so you never need to look at the whole tree if `k` is small.

### Example.

```
Input: root = [5, 3, 6, 2, 4], k = 3
Output: 4
```



**Intuition (diagram).** A BST's **in-order traversal** (left, node, right) visits nodes in strictly increasing order. So the `k`-th smallest is simply the `k`-th node visited in an in-order walk — stop as soon as you've counted `k` nodes instead of walking the whole tree. An **iterative stack** does this without recursion overhead, and lets you bail out early.



```
in-order visit order: 2 -> 3 -> 4 -> 5 -> 6
k value:              1   2   3   4   5
```

### Approach (step by step)

**Key idea / invariant:** simulate the recursive in-order traversal with an explicit stack. The invariant is: whenever we pop a node off the stack, every node with a smaller value has already been visited. So the pop order is exactly the sorted order, and the `count`-th pop is the `count`-th smallest value.

1. Initialize an empty stack and `current = root`, `count = 0`.
2. Loop while `current != null` or the stack is non-empty: a. **Walk left:** push `current` and move `current = current.left` repeatedly, until hitting `null`. This pushes the entire left spine — the path of "smaller than everything above it" candidates. b. **Visit:** pop a node off the stack. This is the next smallest unvisited value. Increment `count`. c. If `count == k`, return this node's value immediately — no need to visit anything else. d. Otherwise, move `current = poppedNode.right` and repeat from step 2a (the right subtree may itself have a left spine of smaller values still to visit).
3. If the loop exits without returning (only possible if `k` was invalid), that's an error case.



### Worked trace on the example tree, $k = 3$ :

```
push spine 5 -> 3 -> 2      stack (top->bottom): [2, 3, 5]
pop 2    -> count=1 (need 3)  current = 2.right = null
pop 3    -> count=2 (need 3)  current = 3.right = 4, push spine: [4]
pop 4    -> count=3 == k      return 4
```

### Brute force

Do a full in-order traversal, collect every value, then index into the list.

```
public int kthSmallestBrute(TreeNode root, int k) {
    List<Integer> values = new ArrayList<>();
    inorder(root, values);
    return values.get(k - 1);
}

private void inorder(TreeNode node, List<Integer> values) {
    if (node == null) return;
    inorder(node.left, values);
    values.add(node.val);
    inorder(node.right, values);
}
```

### Optimized (iterative in-order with a stack)

Walk left as far as possible, pop, count, then step right — stop the moment the count hits  $k$ .

```
public int kthSmallest(TreeNode root, int k) {
    Deque<TreeNode> stack = new ArrayDeque<>();
    TreeNode current = root;
    int count = 0;

    while (current != null || !stack.isEmpty()) {
        while (current != null) {           // push the whole left spine
            stack.push(current);
            current = current.left;
        }
        current = stack.pop();              // visit the smallest unvisited node
        count++;
        if (count == k) {
            return current.val;             // stop early, no need to keep walking
        }
        current = current.right;           // move to the right subtree
    }

    throw new IllegalArgumentException("k is out of range");
}
```

| Version                      | Time                                | Space                                     |
|------------------------------|-------------------------------------|-------------------------------------------|
| Brute force (full traversal) | $O(n)$                              | $O(n)$ for the collected list             |
| Iterative stack, early stop  | $O(k)$ average ( $O(n)$ worst case) | $O(h)$ for the stack ( $h$ = tree height) |

## 4.3 Trapping Rain Water

**Problem.** Given an array `height` of `n` non-negative integers representing an elevation map where each bar has width 1, compute the total volume of water it can trap after it rains (water above index `i` is measured in the same units as `height`).

- **Constraints:**  $1 \leq n \leq 2 \cdot 10^4$ ;  $0 \leq \text{height}[i] \leq 10^5$ . Bars can repeat, be zero, or be strictly increasing/decreasing (which traps no water at all).
- **Clarifying assumptions:** confirm water cannot spill off the two ends (the array boundaries act as implicit walls of height 0 beyond the array) and that `n` can be 1 or 2, in which case the answer is always 0 (need at least one bar on each side to form a "container").
- **Why it's tricky:** the trap is thinking locally — you can't decide how much water sits above bar `i` just by looking at its neighbors; you need the tallest wall *anywhere* to its left and *anywhere* to its right. The optimized solution's insight is that you don't actually need the exact `maxRight` when scanning from the left — you only need to know whether it's *at least* as tall as the current `leftMax`, which two pointers give you for free.

### Example.

```
Input: height = [0,1,0,2,1,0,1,3,2,1,2,1]
Output: 6
```

**Intuition (diagram).** Water sitting above bar `i` is bounded above by  $\min(\text{maxLeft}, \text{maxRight}) - \text{height}[i]$ , where `maxLeft` / `maxRight` are the tallest bars to the left/right of `i` (inclusive). Picture the skyline: water fills in the "valleys" up to the shorter of the two walls surrounding them — `#` is the bar itself, `~` is trapped water.

```
row 3:  . . . . . # . . . .
row 2:  . . . # ~ ~ ~ # # ~ # .
row 1:  . # ~ # # ~ # # # # #
      -- -- -- -- -- -- -- --
height: 0 1 0 2 1 0 1 3 2 1 2 1
index:  0 1 2 3 4 5 6 7 8 9 10 11
```

```
total trapped water = sum of all '~' cells = 6
```

### Approach (step by step)

**Key idea / invariant:** water above index `i` is  $\min(\text{maxLeft}[i], \text{maxRight}[i]) - \text{height}[i]$ , where `maxLeft[i]` is the tallest bar in `height[0..i]` and `maxRight[i]` is the tallest bar in `height[i..n-1]` (both inclusive of `i`). The two-pointer trick avoids precomputing both arrays: walk `lo` from the left and `hi` from the right, tracking `leftMax` and `rightMax` seen so far. The invariant that makes this safe is: **whichever side currently has the smaller running max is the true bottleneck for its pointer's bar** — because the other side is *guaranteed* to have some bar at least that tall between the pointers (otherwise its running max wouldn't be larger), so it can never be the limiting wall. That means it's safe to finalize water at the smaller-max pointer using just its own running max, without ever knowing the exact max on the other side.

1. Set `lo = 0`, `hi = n - 1`, `leftMax = 0`, `rightMax = 0`, `total = 0`.
2. While `lo < hi`: a. If `height[lo] <= height[hi]`, the left side is the bottleneck: update `leftMax = max(leftMax, height[lo])`, add `leftMax - height[lo]` to `total` (this is always  $\geq 0$  since `leftMax >= height[lo]`), then advance `lo++`. b. Otherwise, the right side is the bottleneck: update `rightMax = max(rightMax, height[hi])`, add `rightMax - height[hi]` to `total`, then advance `hi--`.
3. Return `total` once `lo` meets `hi`.

**Worked trace on `height = [0,1,0,2,1,0,1,3,2,1,2,1]`** (indices 0..11):

| step                                                | lo,hi | compare                           | running max             | add             | total | move              |
|-----------------------------------------------------|-------|-----------------------------------|-------------------------|-----------------|-------|-------------------|
| 1                                                   | 0,11  | <code>h[0]=0 &lt;= h[11]=1</code> | <code>leftMax=0</code>  | <code>+0</code> | 0     | <code>lo++</code> |
| 2                                                   | 1,11  | <code>h[1]=1 &lt;= h[11]=1</code> | <code>leftMax=1</code>  | <code>+0</code> | 0     | <code>lo++</code> |
| 3                                                   | 2,11  | <code>h[2]=0 &lt;= h[11]=1</code> | <code>leftMax=1</code>  | <code>+1</code> | 1     | <code>lo++</code> |
| 4                                                   | 3,11  | <code>h[11]=1 &lt; h[3]=2</code>  | <code>rightMax=1</code> | <code>+0</code> | 1     | <code>hi--</code> |
| 5                                                   | 3,10  | <code>h[3]=2 &lt;= h[10]=2</code> | <code>leftMax=2</code>  | <code>+0</code> | 1     | <code>lo++</code> |
| 6                                                   | 4,10  | <code>h[4]=1 &lt;= h[10]=2</code> | <code>leftMax=2</code>  | <code>+1</code> | 2     | <code>lo++</code> |
| 7                                                   | 5,10  | <code>h[5]=0 &lt;= h[10]=2</code> | <code>leftMax=2</code>  | <code>+2</code> | 4     | <code>lo++</code> |
| 8                                                   | 6,10  | <code>h[6]=1 &lt;= h[10]=2</code> | <code>leftMax=2</code>  | <code>+1</code> | 5     | <code>lo++</code> |
| 9                                                   | 7,10  | <code>h[10]=2 &lt; h[7]=3</code>  | <code>rightMax=2</code> | <code>+0</code> | 5     | <code>hi--</code> |
| 10                                                  | 7,9   | <code>h[9]=1 &lt; h[7]=3</code>   | <code>rightMax=2</code> | <code>+1</code> | 6     | <code>hi--</code> |
| 11                                                  | 7,8   | <code>h[8]=2 &lt; h[7]=3</code>   | <code>rightMax=2</code> | <code>+0</code> | 6     | <code>hi--</code> |
| loop ends ( <code>lo=7</code> , <code>hi=7</code> ) |       |                                   |                         | total = 6       |       |                   |

## Brute force

For each bar, scan left and right to find `maxLeft / maxRight`, then add trapped water.

```
public int trapBrute(int[] height) {
    int n = height.length;
    int total = 0;
    for (int i = 0; i < n; i++) {
        int maxLeft = 0;
        for (int l = 0; l <= i; l++) {
            maxLeft = Math.max(maxLeft, height[l]);
        }
        int maxRight = 0;
        for (int r = i; r < n; r++) {
            maxRight = Math.max(maxRight, height[r]);
        }
        total += Math.min(maxLeft, maxRight) - height[i];
    }
    return total;
}
```

## Optimized (two pointers)

Track running maxes from both ends. Whichever side has the smaller running max is the binding constraint, so it's safe to resolve water there and advance that pointer.

```

public int trap(int[] height) {
    int lo = 0, hi = height.length - 1;
    int leftMax = 0, rightMax = 0;
    int total = 0;

    while (lo < hi) {
        if (height[lo] <= height[hi]) {
            leftMax = Math.max(leftMax, height[lo]);
            total += leftMax - height[lo]; // bounded by the smaller wall (left)
            lo++;
        } else {
            rightMax = Math.max(rightMax, height[hi]);
            total += rightMax - height[hi]; // bounded by the smaller wall (right)
            hi--;
        }
    }

    return total;
}

```

| Version      | Time     | Space  |
|--------------|----------|--------|
| Brute force  | $O(n^2)$ | $O(1)$ |
| Two pointers | $O(n)$   | $O(1)$ |

## 4.4 Find All Anagrams in a String

**Problem.** Given two strings `s` and `p`, return a list of all starting indices in `s` where a substring of length `p.length()` is an anagram of `p` (i.e. it has exactly the same letters, possibly in a different order). The output order should match the order the windows occur in `s`.

- **Constraints:**  $1 \leq s.length, p.length \leq 3 * 10^4$ ; both strings consist only of lowercase English letters. `p` can be longer than `s`, in which case the answer is empty.
- **Clarifying assumptions:** confirm the alphabet is restricted to lowercase `a - z` (this is what makes a fixed 26-slot count array viable instead of a general hash map), and that overlapping matches all count (e.g. if two valid windows overlap, both start indices are reported).
- **Why it's tricky:** the naive approach re-sorts or re-counts each window from scratch, paying  $O(m)$  work per window for  $O(n)$  windows. The optimization requires noticing that consecutive windows overlap in all but 2 characters, so counts can be updated incrementally in  $O(1)$  per step instead of  $O(m)$ .

### Example.

```

Input: s = "cbaebabacd", p = "abc"
Output: [0, 6]
("cba" at index 0 and "bac" at index 6 are anagrams of "abc")

```

**Intuition (diagram).** Every anagram of `p` has the exact same letter counts as `p`, just reordered. Slide a **fixed-size window** (length = `p.length()`) across `s`, keeping a running count of the 26 letters inside it.

When the window's counts match `p`'s counts exactly, its start index is an answer. Sliding the window is  $O(1)$  per step: drop the letter leaving, add the letter entering.

```
index:  0  1  2  3  4  5  6  7  8  9
s:      c  b  a  e  b  a  b  a  c  d

win@0:   c  b  a  .  .  .  .  .  .  .  -> counts {a:1,b:1,c:1} == p -> match, index 0
win@1:   .  b  a  e  .  .  .  .  .  .  -> counts {a:1,b:1,e:1} != p -> no
...      (windows 2..5 also fail; omitted for brevity)
win@6:   .  .  .  .  .  b  a  c  .  .  -> counts {a:1,b:1,c:1} == p -> match, index 6
```

```
p = "abc" -> need counts: a:1 b:1 c:1
```

## Approach (step by step)

**Key idea / invariant:** maintain a 26-slot `window` count array that always reflects exactly the letters currently inside the last `m` characters seen (`m = p.length()`), where each pointer step adds the incoming character and — once the window is full — removes the character falling off the back. The window only ever grows to size `m` and then slides, so each of the `n` steps does  $O(1)$  array work, and comparing two 26-slot arrays is  $O(26) = O(1)$ .

- Let `n = s.length()`, `m = p.length()`. If `m > n`, return an empty result immediately — no window fits.
- Build `need[26]`, the letter-count histogram of `p`.
- Initialize `window[26]` to all zeros. Iterate `i` from `0` to `n - 1`: a. **Add** the incoming character: `window[s.charAt(i) - 'a']++`. b. **Evict** once the window would exceed size `m`: if `i >= m`, decrement the count for `s.charAt(i - m)` — the character that just fell out of the back of the window. c. **Check** once the window has reached full size `m` (i.e. `i >= m - 1`): if `window` equals `need` exactly, the window starting at `i - m + 1` is an anagram — record that start index.
- Return the collected list of start indices.

**Worked trace on `s = "cbaebabacd"`, `p = "abc"` (`m = 3`, `need = {a:1,b:1,c:1}`):**

```
i=0: add 'c'          window={c:1}          (window size 1, no check yet)
i=1: add 'b'          window={c:1,b:1}       (window size 2, no check yet)
i=2: add 'a'          window={c:1,b:1,a:1} == need -> match at start 2-3+1=0
i=3: add 'e'; evict s[0]='c' window={b:1,a:1,e:1} != need -> no
i=4: add 'b'; evict s[1]='b' window={a:1,e:1,b:1} != need -> no
i=5: add 'a'; evict s[2]='a' window={e:1,b:1,a:1} != need -> no
i=6: add 'b'; evict s[3]='e' window={b:2,a:1} != need -> no
i=7: add 'a'; evict s[4]='b' window={b:1,a:2} != need -> no
i=8: add 'c'; evict s[5]='a' window={b:1,a:1,c:1} == need -> match at start 8-3+1=6
i=9: add 'd'; evict s[6]='b' window={a:1,c:1,d:1} != need -> no
result: [0, 6]
```

## Brute force

For every window of `p`'s length, sort both and compare.

```

public List<Integer> findAnagramsBrute(String s, String p) {
    List<Integer> result = new ArrayList<>();
    int n = s.length(), m = p.length();
    char[] target = p.toCharArray();
    Arrays.sort(target);
    String sortedP = new String(target);

    for (int i = 0; i + m <= n; i++) {
        char[] window = s.substring(i, i + m).toCharArray();
        Arrays.sort(window);
        if (new String(window).equals(sortedP)) {
            result.add(i);
        }
    }
    return result;
}

```

## Optimized (fixed sliding window of 26 counts)

Maintain a count array for the window and one for `p`; compare in  $O(26)$  instead of sorting.

```

public List<Integer> findAnagrams(String s, String p) {
    List<Integer> result = new ArrayList<>();
    int n = s.length(), m = p.length();
    if (m > n) return result;

    int[] need = new int[26];
    int[] window = new int[26];
    for (char c : p.toCharArray()) {
        need[c - 'a']++;
    }

    for (int i = 0; i < n; i++) {
        window[s.charAt(i) - 'a']++;           // add the incoming letter
        if (i >= m) {
            window[s.charAt(i - m) - 'a']--;   // drop the letter leaving the window
        }
        if (i >= m - 1 && Arrays.equals(window, need)) {
            result.add(i - m + 1);             // window start index
        }
    }
    return result;
}

```

| Version                        | Time              | Space                   |
|--------------------------------|-------------------|-------------------------|
| Brute force (sort each window) | $O(n * m \log m)$ | $O(m)$                  |
| Fixed sliding window of counts | $O(n)$            | $O(1)$ (26-slot arrays) |

## Closing notes

- If the answer to a problem is monotonic (true/false or bigger/smaller flips exactly once as some parameter grows), that's your cue to binary search on the answer instead of the input.

- If you're going to query the same set of strings/prefixes many times, build a trie once up front rather than re-scanning strings per query.
  - Before writing any DP code, say the recurrence out loud in plain words — if you can't state it in a sentence, you don't understand the subproblem yet.
  - Always state the time and space complexity out loud, even if the interviewer doesn't ask — it shows you're thinking about it, and it often surfaces bugs in your own approach.
  - Trace your optimized solution through the worked example by hand before you say "done" — it catches off-by-one errors in windows, pointers, and indices.
  - Explicitly handle edge cases: empty input, a single element, all-duplicate values, and null nodes/pointers — interviewers watch for these even when they don't say so.
-